EIMS graduation report

Evidence grounded multi vendor log investigation

Ratthabhumi Peansukmanee
Computer Engineering International Program
King Mongkut’s Institute of Technology Ladkrabang
Academic year 2026

Staged assembly for author and institutional review. This copy preserves current graduation revisions and explicitly marked historical prototype chapters. It is not an approved thesis or a production certification.

Evidence and publication boundaries

Analyzer publication baseline 2a0d903: 391 engineering tests. Verification baseline 7335aa2: 394 engineering tests, CI 41308596046. RC2 changes only the shared investigation presentation and delivery artifacts; final source SHA and CI are recorded in PR #10 and the staging manifest. Scientific baseline remains a7b5291: 12 DRAFT DEV, 61 claims, zero genuine human review. Diagnostic accuracy is NOT_MEASURED.

HTTPS browser acceptance remains blocked by authorization. Physical USB and production are not tested. The September observations do not certify this release. Institutional cover, approval/signature pages and author biography require owner-supplied forms; no signatures or approval are invented. Mermaid diagrams are preserved as source listings pending author conversion to approved figures.

Contents

Evidence and publication boundaries 1
Contents 2
Current abstract revision — 31 October 2026 7
Historical September abstract 7
บทคัดย่อ 7
Current abstract revision — 31 October 2026 9
Historical September abstract 9
Abstract 9
Acknowledgments 11
Chapter 1 12
Introduction 12
Current graduation revision and evidence boundary 12
Historical prototype material (retained for provenance) 12
1.1 Background and Significance 12
1.2 Problem Statement 13
1.3 Project Objectives 14
1.4 Project Scope 15
In Scope 15
Out of Scope and Design Boundaries 16
1.5 Project Methodology Overview 16
1.6 Expected Benefits 17
1.7 Chapter Summary 17
Chapter 2 18
Concepts and Related Technologies 18
Current graduation revision and evidence boundary 18
Historical prototype material (retained for provenance) 18
2.1 Infrastructure Asset Management 18
2.2 Endpoint Evidence Collection 19
2.3 Windows Management Technologies 20
2.4 Windows Event Logging 21
Event Log Structural Hierarchy 21
Operational Channel Selection 21

2.5 Evidence Provenance and Deduplication 22
Deduplication Architecture: Application-Level vs. Database Constraints 22
2.6 Event Prioritization 23
2.7 AI-assisted Log Analysis 23
Semantic Vector Embeddings 23
Retrieval-Augmented Generation (RAG) Architecture 24
2.8 Relational and Semi-Structured Data Storage 24
PostgreSQL Relational Datastore 24
Handling Polymorphic Telemetry via JSONB 24
Vector Persistence via pgvector 25
2.9 Redis and Asynchronous Processing 25
2.10 Object Storage and Optical Character Recognition (OCR) 25
2.11 Backend API Framework 26
2.12 Frontend Architecture 26
Hydration as an Architectural Consideration 27
2.13 Authentication and Security Boundary 27
Authentication Modes: Demo vs. Secure Architecture 27
Server-Side Administrative Boundary 27
2.14 Observability Technologies 27
2.15 Related Engineering Approaches 28
2.16 Chapter Summary 28
Chapter 3 30
System Design and Implementation 30
Current graduation revision and evidence boundary 30
Historical prototype material (retained for provenance) 30
3.1 Requirement Analysis 30
3.1.1 Operational Personas 30
3.1.2 Canonical Architecture & Target Requirements (PRD Baseline) 31
Canonical Functional Requirements 31
3.1.3 Canonical Non-Functional Requirements 32
3.1.4 Implemented Graduation System Scope 33
3.2 Overall System Architecture 33
Major Architectural Components 33
System Architecture Diagram 34

3.3 Asset Registry and Data Model 35
Core Relational Entities 35
Entity Relationship Diagram 36
3.4 Portable USB Auditor Design 37
Core Architectural Decisions 37
Builder Safety Controls 38
3.5 Endpoint Evidence Collection 38
Event Collection Constraints 39
3.6 Offline Report Schema and Ingestion 39
End-to-End Ingestion Flow 39
Offline Evidence Ingestion Sequence 39
3.7 Sequential Deduplication Design 40
Deduplication Key Derivation 40
Query Evaluation and Filtering 40
Architectural Limitation and Concurrency Boundary 41
3.8 Event Prioritization 41
3.9 AI-assisted Analyzer Integration 41
Semantic Retrieval Pipeline 41
3.10 Analysis Provenance 42
3.11 Backend API Design 42
Core Interface Endpoints 42
Error Handling Architecture 43
3.12 Investigation Dashboard 43
Interface Views 43
3.13 Authentication and Security Design 44
Dual Authentication Modes 44
Removal of Privileged Static Tokens 44
Trusted Local Sandbox Limitation 44
3.14 Observability and Supporting Infrastructure 44
3.15 Development and Validation Methodology 45
3.16 Data Protection and Evidence Handling 45
3.17 Design Limitations 46
3.18 Chapter Summary 46
Chapter 4 48

Results and Validation 48
Current graduation revision and evidence boundary 48
Current October engineering and scientific results 48
Historical prototype material (retained for provenance) 48
4.1 Validation Objectives 48
4.2 Validation Environment 49
4.3 Automated Regression and Build Validation 50
4.4 Portable Evidence Collection Validation 50
4.5 Asset Registry and Compliance Results 51
4.6 Windows Event Evidence Results 51
Observed Field Metrics 52
4.7 Sequential Deduplication Validation 52
Previously Validated Ingestion Behavior 52
Operational Boundaries 53
4.8 Event Prioritization and Analyzer Results 53
Prioritization Rule Execution 53
Field Analysis Findings 53
4.9 AnalysisHistory and Provenance Results 53
Observed Metrics and Query Integrity 53
Provenance Attribute Verification 54
4.10 Endpoint Investigation UI Validation — A9 Gate 54
Verification Criteria and Observed Results 54
4.11 Supporting Search and Observability Results 55
Core Service Health 55
Status of Secondary Supporting Capabilities 55
4.12 Defects Identified and Corrected During Validation 55
4.13 Evidence Handling and Confidentiality 56
4.14 Validation Limitations 57
4.15 Validation Summary 57
Chapter 5 59
Summary and Recommendations 59
Current graduation revision and evidence boundary 59
Historical prototype material (retained for provenance) 59
5.1 Project Summary 59

5.2 Achievement of Project Objectives 60
5.3 Key Engineering Outcomes 62
1. Unified Infrastructure Evidence Representation 62
2. Zero-Dependency Portable Evidence Collection 62
3. Traceable Event-to-Analysis Provenance Chain 62
4. Application-Level Duplicate Protection 63
5. Workload-Bounded AI Diagnostic Assistance 63
6. Real-World Field Validation on Production Infrastructure 63
5.4 Validation Summary 63
5.5 Engineering Challenges and Lessons Learned 64
1. Server-Side Rendering (SSR) and Client Hydration Discrepancies 64
2. Framework Annotation Typing Bugs in Dynamic Routers 64
3. Dataset Provenance and Multi-Environment Discrepancies 64
4. Database Safety Controls and Benchmark Isolation 65
5. Multi-Tier Evidence-Driven Verification 65
5.6 Current System Limitations 65
5.7 Recommendations and Future Work 66
1. Concurrency-Safe Deduplication and Relational Hardening 66
2. High Availability and Enterprise Scale Production Clustering (Roadmap Milestone 1) 66
3. Cross-Endpoint Anomaly Correlation (Roadmap Milestone 2) 66
4. Enterprise Identity Federation and Granular RBAC (Roadmap Milestone 2) 67
5. Multi-Platform Endpoint Expansion (Possible Future Extension) 67
6. Rigorous Long-Duration Operational Validation 67
5.8 Overall Conclusion 67
References 69

Source docs/report/ABSTRACT_TH.md

Current abstract revision — 31 October 2026

Current source audit: 1 November 2026 — 2a0d9031297ac1063a0f327a8a76c455bb5da315 (PR #10). Next.js 16.3.0 / React 19.2.8. Engineering CI: 391 Python tests (299 Analyzer hermetic + 46 synthetic PostgreSQL + 46 USB isolated), run 41230498604, not project-wide certification. DEV 12 DRAFT, HELDOUT 5 reserved, claims 61, genuine human reviews 0. Diagnostic accuracy NOT_MEASURED; secure HTTPS browser BLOCKED_NOT_TESTED; current USB/production acceptance NOT_TESTED. No main merge.

ระบบหลักรุ่นปัจจุบันวิเคราะห์ log หลายผู้ผลิตด้วยกฎและหลักฐาน แยกอาการจากสมมติฐานและสาเหตุที่ยังไม่ทราบ พร้อม History แยกเจ้าของ ผล M0/M1/Full ใช้ DEV 12 เคสที่ยังเป็น DRAFT และ Full มี 61 claims ที่ยังไม่มีคนตรวจ จึงยังวัด Accuracy ไม่ได้ ผลทดสอบวิศวกรรมไม่ใช่ผลรับรองคำวินิจฉัย ส่วน HTTPS browser และการติดตั้งจริงรุ่นล่าสุดยังไม่ผ่านการตรวจรับ บทคัดย่อเดิมด้านล่างเป็นหลักฐานรุ่นก้นยายน ต้องไม่ใช้แทนบทคัดย่อปัจจุบันโดยละข้อความปรับปรุงนี้

Historical September abstract

บทคัดย่อ

การบริหารจัดการโครงสร้างพื้นฐานเทคโนโลยีสารสนเทศในเครือข่ายองค์กรที่มีการแบ่งส่วนการเชื่อมต่อ มักประสบปัญหาความกระจัดกระจายของอุปกรณ์ประมวลผล สภาพแวดล้อมเครือข่ายที่มีการแยกขาด (Air-gapped Network) และข้อมูลการทำงานของระบบที่กระจัดกระจาย เมื่อเกิดความขัดข้องขึ้นบนอุปกรณ์ปลายทางที่ไม่มีการติดตั้งโปรแกรมตรวจติดตามแบบต่อเนื่อง ผู้ดูแลระบบจำเป็นต้องแก้ไขปัญหาแบบเฉพาะหน้า ซึ่งใช้ระยะเวลานานและขาดความสามารถในการตรวจสอบย้อนกลับ โครงงานนี้นำเสนอระบบบริหารจัดการโครงสร้างพื้นฐานองค์กร (Enterprise Infrastructure Management System: EIMS) ซึ่งเป็นแพลตฟอร์มบริหารจัดการข้อมูลหลักฐานด้านโครงสร้างพื้นฐานและการวิเคราะห์เพื่อสนับสนุนการตรวจสอบด้วยปัญญาประดิษฐ์ (AI-assisted Investigation) ที่ออกแบบขึ้นเพื่อผสานการทำงานระหว่างเครือข่ายหลักและอุปกรณ์ปลายทางที่ไม่ได้เชื่อมต่อเครือข่าย ระบบ EIMS ทำงานร่วมกันระหว่างทะเบียนทรัพย์สินโครงสร้างพื้นฐานบนฐานข้อมูล PostgreSQL และชุดโปรแกรมรวบรวมข้อมูลหลักฐานแบบพกพาที่พัฒนาขึ้นบน CPython 3.14.3 ชนิดฝังตัว (Embedded Runtime) โดยชุดรวบรวมข้อมูลจะเรียกค้นข้อมูลระบบด้วย Windows Management Instrumentation (WMI) แบบไม่รบกวนการทำงานของระบบ และคัดกรองข้อมูลเหตุการณ์ของระบบ Windows เฉพาะช่องทาง System และ Application ในกรอบเวลา 24 ชั่วโมง โดยกำหนดขีดจำกัดเริ่มต้นไว้ที่ 500 เหตุการณ์สำหรับการสืบค้นรวม จัดเก็บเป็นแฟ้มข้อมูล JSON ออฟไลน์ที่มีการตรวจสอบโครงสร้างข้อมูลอย่างรัดกุม เมื่อนำเข้าข้อมูลสู่ส่วนกลาง ระบบจะดำเนินการตรวจสอบข้อมูลซ้ำในระดับแอปพลิเคชันสำหรับการนำเข้าทีละชุดด้วยการสร้างค่าแฮช SHA-256 ของแต่ละเหตุการณ์ พร้อมทั้งจัดลำดับความสำคัญตามระดับความรุนแรง ในขั้นตอนการวิเคราะห์เพื่อช่วยสนับสนุนการวินิจฉัยปัญหาของเจ้าหน้าที่ ตัววิเคราะห์ปัญญาประดิษฐ์จะถูกจำกัดขอบเขตการทำงานไว้ที่ไม่เกิน 10 รายการต่อรอบการนำเข้า โดยประยุกต์ใช้เวกเตอร์การฝังความหมายของข้อความ (FastEmbed BAAI/bge-small-en-v1.5 ร่วมกับ SentenceTransformer all-MiniLM-L6-v2 สำรอง) ร่วมกับสถาปัตยกรรมการสืบค้นเสริมการสร้างข้อความ (RAG) และบันทึกผลการวินิจฉัยเป็นประวัติผลการวิเคราะห์ (AnalysisHistory) ที่เชื่อมโยงย้อนกลับไปยังทรัพย์สินนั้น การประเมินผลระบบครอบคลุมชุดการทดสอบอัตโนมัติ การนำเข้าข้อมูลภาคสนามในอดีต และการตรวจสอบความถูกต้องของขั้นตอนหลังการนำเข้าข้อมูล โดยชุดการทดสอบการนำเข้าข้อมูลแบบแยกส่วนของ Sprint 13 ผ่านการทดสอบ 37 รายการ (ล้มเหลว 0 รายการ คำเตือน 4 รายการ) ส่วนต่อประสานผู้ใช้ผ่านการตรวจสอบชนิดข้อมูลของ TypeScript และการสร้างแพ็กเกจสำหรับใช้งานจริง ชุดข้อมูลภาคสนามยืนยันการจัดเก็บข้อมูลเหตุการณ์ของระบบ Windows จำนวน 49 รายการ (System 34 รายการ, Application 15 รายการ) และมีการบันทึกประวัติผลการวิเคราะห์ 10 รายการสำหรับเครื่องเป้าหมาย (ASSET-01) ส่วนต่อประสานโปรแกรมประยุกต์ (API) สำหรับคัดกรองประวัติการวิเคราะห์ตอบสนองด้วยรหัสสถานะ HTTP 200 และสามารถแสดงข้อมูลเหตุการณ์และผลการวิเคราะห์บนหน้าเว็บได้ตามที่ทดสอบ (A9 PASS) ทั้งนี้ การทดสอบภาคสนามก่อนหน้าได้พิสูจน์กระบวนการเก็บรวบรวมข้อมูลด้วยอุปกรณ์พกพาและการนำเข้าข้อมูลตามลำดับ

ในขณะที่การตรวจสอบหน้างานครั้งล่าสุดเป็นการยืนยันความถูกต้องของการจัดเก็บข้อมูลหลักฐานในฐานข้อมูล ประวัติการวิเคราะห์ การตอบสนองของ API แบบคัดกรอง และการแสดงผลบนเว็บเบราว์เซอร์ ระบบ EIMS
แสดงให้เห็นถึงความเป็นไปได้ทางวิศวกรรมในการบริหารจัดการข้อมูลหลักฐานที่สามารถตรวจสอบย้อนกลับได้
และการวิเคราะห์สนับสนุนด้วยปัญญาประดิษฐ์สำหรับระบบออฟไลน์ ภายใต้ขอบเขตการเป็นระบบต้นแบบเชิงวิศวกรรมสำหรับรายงานสหกิจศึกษา
ซึ่งมีความแตกต่างจากระบบเชิงพาณิชย์สำหรับสภาพแวดล้อมการผลิตจริงหรือระบบนิติวิทยาศาสตร์ดิจิทัล

คำสำคัญ: ระบบบริหารจัดการโครงสร้างพื้นฐานองค์กร, ข้อมูลหลักฐานด้านโครงสร้างพื้นฐาน, บันทึกเหตุการณ์ระบบ Windows,
การรวบรวมข้อมูลหลักฐานแบบออฟไลน์, การวิเคราะห์เพื่อสนับสนุนการตรวจสอบด้วยปัญญาประดิษฐ์, การจัดการทรัพย์สิน, การสังเกตการณ์ระบบ

Source docs/report/ABSTRACT_EN.md

Current abstract revision — 31 October 2026

Current source audit: 1 November 2026 — 2a0d9031297ac1063a0f327a8a76c455bb5da315 (PR #10). Next.js 16.3.0 / React 19.2.8. Engineering CI: 391 Python tests (299 Analyzer hermetic + 46 synthetic PostgreSQL + 46 USB isolated), run 41230498604, not project-wide certification. DEV 12 DRAFT, HELDOUT 5 reserved, claims 61, genuine human reviews 0. Diagnostic accuracy NOT_MEASURED; secure HTTPS browser BLOCKED_NOT_TESTED; current USB/production acceptance NOT_TESTED. No main merge.

The current graduation core performs deterministic multi-vendor investigation with evidence-linked findings, conservative causal uncertainty and authenticated user-owned History. Reproducible M0/M1/Full evaluations execute 12 DRAFT DEV cases; 61 Full claims have no genuine human judgments, so diagnostic accuracy is not measured. Automated engineering tests and historical field ingestion support different, limited claims. Secure HTTPS browser and latest physical installation acceptance remain pending. The historical abstract below describes the September prototype and is retained for traceability; it must not be submitted as the current abstract without this revision.

Historical September abstract

Abstract

Enterprise infrastructure management across segmented corporate networks is frequently challenged by distributed compute assets, air-gapped network enclaves, and fragmented operational evidence. When infrastructure failures occur on hosts lacking persistent monitoring agents, administrators face labor-intensive, ad-hoc diagnostics that lack structured traceability. This project presents the Enterprise Infrastructure Management System (EIMS), an integrated infrastructure evidence management and artificial intelligence (AI)-assisted investigation platform designed to bridge connected operations and air-gapped endpoints. EIMS combines a centralized PostgreSQL asset registry with an episodic, portable evidence collection utility built on an embedded CPython 3.14.3 runtime. The collector executes non-destructive Windows Management Instrumentation (WMI) queries and extracts Windows Event Log records, specifically scoped to the System and Application channels over a 24-hour window with a default maximum of 500 events across the combined query, into structured, schema-validated offline JSON artifacts. Upon central ingestion, EIMS enforces application-level sequential deduplication via SHA-256 event fingerprinting and prioritizes critical occurrences. To accelerate administrative triage without autonomous overreach, a bounded AI analyzer evaluates at most ten prioritized events per import using semantic embeddings (FastEmbed BAAI/bge-small-en-v1.5, with SentenceTransformer all-MiniLM-L6-v2 fallback) and retrieval-augmented generation, persisting diagnostic findings as auditable AnalysisHistory records linked to the asset. System verification encompassed automated testing, historical field imports, and live downstream validation. The hermetic Sprint 13 ingestion test suite achieved 37 passed tests (0 failed, 4 warnings), while frontend TypeScript and production builds executed cleanly. The field-validated dataset confirmed 49 persisted Windows Event Evidence records (34 System, 15 Application), yielding 10 bounded AnalysisHistory records for the evaluated host (ASSET-01). The asset-filtered history API returned an HTTP 200 response, and live browser testing successfully validated the end-to-end presentation of event evidence and AI findings (A9 PASS). Previous field validation demonstrated the portable acquisition and sequential import workflow, while the latest on-site verification confirmed the downstream persisted evidence, analysis history, filtered API,

and browser presentation. EIMS proves the engineering feasibility of traceable, air-gapped evidence management and assistive AI diagnostics within a well-defined academic prototype baseline, distinct from a production-hardened or forensic system.

Keywords: Enterprise Infrastructure Management, Infrastructure Evidence, Windows Event Log, Offline Evidence Collection, AI-assisted Investigation, Asset Management, System Observability

Source docs/report/ACKNOWLEDGMENTS.md

Acknowledgments

This cooperative education project, titled "Development of an Enterprise Infrastructure Management System (EIMS)," was conducted as part of the undergraduate curriculum in the Department of Computer Engineering, Faculty of Engineering, King Mongkut's Institute of Technology Ladkrabang (KMITL). The author wishes to express sincere gratitude to all individuals and organizations whose support, mentorship, and technical guidance made the successful realization of this engineering work possible.

Foremost, the author expresses deep appreciation to KANEMATSU ELECTRONICS (THAILAND) for providing the invaluable opportunity to undertake this cooperative education placement, as well as access to enterprise engineering facilities, infrastructure environments, and operational resources necessary for system development and validation.

The author extends heartfelt thanks to Mr. Chalermpol Phuangchalam, workplace supervisor, along with the engineering mentors and technical team members at KANEMATSU ELECTRONICS (THAILAND). Their continuous technical guidance, constructive code reviews, domain insights into enterprise systems administration, and practical feedback throughout the iterative development sprints contributed fundamentally to the quality and engineering value of this project.

Special gratitude is offered to Asst. Prof. Dr. Phongsak Keeratiwintakorn, cooperative education academic advisor, and the faculty members of the Department of Computer Engineering at KMITL for their academic guidance, methodological advice, and thorough oversight throughout the project lifecycle. Finally, the author wishes to thank family and colleagues for their constant encouragement and understanding during the course of this study.

Source docs/report/CHAPTER_1_INTRODUCTION.md

Chapter 1

Introduction

Current graduation revision and evidence boundary

Current source audit: 1 November 2026 — 2a0d9031297ac1063a0f327a8a76c455bb5da315 (PR #10). Next.js 16.3.0 / React 19.2.8. Engineering CI: 391 Python tests (299 Analyzer hermetic + 46 synthetic PostgreSQL + 46 USB isolated), run 41230498604, not project-wide certification. DEV 12 DRAFT, HELDOUT 5 reserved, claims 61, genuine human reviews 0. Diagnostic accuracy NOT_MEASURED; secure HTTPS browser BLOCKED_NOT_TESTED; current USB/production acceptance NOT_TESTED. No main merge.

The current graduation objective is evidence-grounded, multi-vendor log investigation with reproducible evaluation. Asset inventory and portable Windows ingestion remain implemented supporting modules. The October core distinguishes event identity, observed technical failure, explicit terminal status, causal hypotheses and UNKNOWN physical cause. Acceptance requires authentic reviewed-only judgments and an independently authorized HTTPS demonstration, not simply more features. The objectives and field workflow below document the earlier prototype; the RAG objective is not an executed M2/M3 experiment in the current evaluation.

Historical prototype material (retained for provenance)

1.1 Background and Significance

In contemporary enterprise computing environments, managing, auditing, and troubleshooting distributed infrastructure presents significant operational challenges. Enterprise organizations frequently deploy heterogeneous computing estates comprising mission-critical database servers, directory controllers, specialized industrial workstations, and departmental endpoints. While automated centralized monitoring solutions exist for well-connected corporate networks, modern enterprise topologies frequently feature segmented network architectures, demilitarized zones (DMZs), isolated operational technology (OT) subnets, and air-gapped environments. In these environments, continuous outbound connectivity to centralized cloud monitoring or Software-as-a-Service (SaaS) management platforms is often constrained by organizational security policies, strict change-control procedures, or physical network isolation.

When operational anomalies, security policy infractions, or hardware degradations manifest on disconnected, semi-isolated, or policy-restricted endpoints, system administrators and IT infrastructure engineers are confronted with difficulty investigating faults. Diagnostic information remains fragmented across disparate utilities and local files and data stores: hardware inventory details are manually compiled in spreadsheets, system logs are locked inside local Windows Event Log binary files, and security posture evaluations are conducted using manual, ad-hoc checklist audits. Furthermore, the operational overhead associated with dispatching field technicians to manually execute diagnostic commands on unmanaged endpoints results in longer troubleshooting time, undocumented configuration drift, and elevated vulnerability windows.

These operational challenges motivated the development of EIMS to support evidence collection across both connected and disconnected endpoints. Such a platform must accommodate flexible evidence collection workflows, support portable data acquisition where persistent agents cannot be deployed, preserve traceable data provenance, and provide structured analytical capabilities. Furthermore, given the immense volume and syntactic complexity of native operating system event streams, integrating artificial intelligence (AI) and vector retrieval mechanisms as assistive triage tools offers substantial potential to augment human systems engineers, expediting operational investigation without replacing human operational oversight.

The Enterprise Infrastructure Management System (EIMS) was designed and implemented to address this operational demand. EIMS serves as an infrastructure evidence management and AI-assisted investigation platform. It provides a cohesive architecture for registering enterprise assets, acquiring operational and security evidence from connected or offline endpoints, normalizing heterogeneous telemetry into structured relational datastores, preserving traceable data provenance, and delivering context-aware diagnostic assistance through semantic vector search and modern web visualization.

1.2 Problem Statement

Infrastructure administration and incident investigation in enterprise environments are impeded by six core engineering problems:

1. Fragmentation of Infrastructure Information Across Disparate Repositories:

Hardware specifications, network interface allocations, operating system patch levels, local security configurations, and operational logs are stored in disparate formats across multiple disconnected utilities. Systems administrators lack a single, consolidated repository that correlates physical and logical asset identity with current operational health and historical state.

2. Operational Constraints Limiting Persistent Daemon Deployment:

In air-gapped servers, high-security financial subnets, and temporary field machines, continuous outbound network connections to centralized management platforms are frequently disallowed. Furthermore, enterprise change-control policies often restrict installing persistent third-party background services or external runtime dependencies (such as Python interpreters, package managers, or container engines) on mission-critical servers solely for periodic maintenance or auditing.

3. Risk of Duplicate Operational Records During Sequential Evidence Ingestion:

When diagnostic data is gathered periodically using portable storage media and subsequently uploaded to a centralized platform, repeated uploads of overlapping time-window reports inevitably occur. Without deterministic deduplication mechanisms, identical event records are ingested repeatedly, polluting operational databases, inflating storage consumption, and distorting historical event frequency metrics.

4. High Volume and Low Signal-to-Noise Ratio in Operating System Event Streams:

Native operating system event logs, specifically Microsoft Windows Event Logs, generate tens of thousands of operational records daily. The vast majority of these events represent routine informational status notifications. Identifying, filtering, and prioritizing critical failure indicators and warning anomalies within this high-velocity stream requires significant cognitive effort, creating substantial triage latency for systems operators.

5. Absence of Traceable Diagnostic Provenance Across Ingested Findings:

When automated diagnostic tools or artificial intelligence models generate remediation advice or anomalous incident flags, operators often cannot trace the recommendation back to the specific originating event, source file, ingestion batch, or host machine. In enterprise auditing, recommendations that lack clear provenance cannot be verified or trusted during incident post-mortems.

6. Absence of Unified Investigation Interfaces and Historical Search:

IT operations teams frequently switch between command-line consoles, local event viewers, database management tools, and web portals to conduct cross-domain investigations. The absence of a unified web interface providing keyboard-driven global search, synchronized chronological timelines, and asset-linked diagnostic summaries severely constrains operator efficiency during time-sensitive service disruptions.

1.3 Project Objectives

To resolve the identified engineering problems, the development of EIMS was guided by seven concrete engineering objectives:

1. Architect and Implement a Centralized Asset Registry:

Design a normalized relational persistence model and API gateway capable of indexing enterprise infrastructure assets, cataloging hardware inventories, tracking network configurations, and maintaining lifecycle states.

2. Develop a Lightweight, Portable Endpoint Collection Utility:

Implement a portable auditing package (the USB Auditor) containing an embedded CPython 3.14.3 amd64 runtime environment that executes on target Windows hosts without requiring preinstalled dependencies, privileged software installers, or persistent background daemons, emitting structured JSON evidence payloads.

3. Establish an Offline Ingestion and Normalization Pipeline:

Construct backend API endpoints and ingestion workflows capable of parsing offline multipart JSON evidence reports, updating asset entity attributes, and persisting operating system events into structured database tables.

4. Implement Deterministic Sequential Deduplication and Severity Prioritization:

Design an application-level cryptographic fingerprinting algorithm using SHA-256 hashes to prevent redundant record insertion during sequential report imports, coupled with a severity-ranked priority queue to isolate high-impact operational events.

5. Integrate an AI-Assisted Log Investigation Engine:

Incorporate semantic vector retrieval using FastEmbed (primary BAAI/bge-small-en-v1.5, with SentenceTransformer all-MiniLM-L6-v2 fallback) and PostgreSQL pgvector similarity search to match incoming operational anomalies against known solutions, producing structured diagnostic summaries and mitigation recommendations.

6. Deliver an Interactive Web-Based Operational Dashboard:

Develop a responsive, modern web interface utilizing Next.js, React, and TypeScript that renders asset details, security and compliance postures, event evidence tables, AI findings, a multi-domain global search palette, and unified chronological timelines.

7. Validate System Reliability and Integrity Through Automated and Field Testing:

Verify platform stability, data provenance, and operational workflows using automated unit and integration test suites, schema validation, type checking, and real-world field validation across authorized Windows Server and workstation endpoints.

1.4 Project Scope

To define the scope within the cooperative education framework, the capabilities of EIMS are explicitly divided into implemented functional features and external boundaries.

In Scope

- Target Operating Environment: Windows-first enterprise infrastructure, specifically targeting Windows Server and workstation endpoints (with field validation conducted on an authorized Windows Server host).

- Centralized Asset Management: Relational asset registry maintaining canonical host identity, IP addresses, hardware component inventories, and operational lifecycle state transitions.

- Portable Offline Auditor: A standalone, USB-deployable auditing package featuring an embedded CPython 3.14.3 amd64 runtime, launched via automated batch scripts, producing standardized local JSON reports while maintaining a strictly non-destructive, read-only operational posture.

- Operating System Telemetry Ingestion: Structured acquisition and parsing of Windows Event Logs. The portable graduation workflow intentionally scopes Windows Event Log collection to the System and Application channels across configurable historical time windows (defaulting to 24 hours, with a default maximum of 500 events across the combined System and Application query, filtering Critical, Error, and Warning events).

- Endpoint Security and Compliance Auditing: Algorithmic evaluation of endpoint security configurations, assessing Windows Defender status, Windows Firewall enforcement, Windows Update recency, and BitLocker encryption status (strictly constrained to posture state without collecting recovery keys).

- Supporting Optical Character Recognition (OCR): Secondary ingestion pipeline accepting multipart physical hardware manifests, invoices, or chassis specification labels, storing raw binaries in S3-compatible object storage (MinIO) and extracting serial metadata to assist asset onboarding.

- AI-Assisted Investigation Pipeline: Local semantic embedding generation (384-dimensional vectors) and vector database cosine distance querying to retrieve diagnostic solutions for prioritized events, bounded to an operational workload limit of at most 10 analyzed events per import.

- Audit and Provenance Tracking: Traceable linkage between asset entities, persisted event logs, and analysis history metadata, including source_type classification (USB_OFFLINE_COLLECTION) and event-level deduplication metadata.

- Web Investigation Portal: Centralized web dashboard providing asset inventory navigation, detailed endpoint inspection, global keyboard search (Ctrl+K), unified multi-domain timelines, and diagnostic visualization.

- Prototype Deployment Architecture: Containerized multi-service deployment orchestrating FastAPI, PostgreSQL with pgvector, Redis, PgBouncer, MinIO, Prometheus, and Grafana using Docker Compose.

Out of Scope and Design Boundaries

- Certified Digital Forensics: EIMS is an IT operational triage and management platform; it is not designed, validated, or certified as a court-admissible forensic acquisition utility or cryptographic chain-of-custody evidence vault.

- Production High-Availability (HA) Clustering: The prototype deployment utilizes single-instance container services; multi-region active-active database clustering, automatic failover controllers, and Redis Sentinel topologies are not implemented in the current baseline.

- Enterprise Distributed Orchestration: Deployment via Kubernetes, Helm charts, or cloud-managed container services remains out of scope for the current cooperative education release.

- Enterprise Identity Federation: Integration with enterprise Single Sign-On (SSO) protocols, such as SAML 2.0, OpenID Connect (OIDC) identity brokers, or direct Active Directory Domain Controller replacement, is excluded.

- Autonomous Remote Fleet Management: EIMS does not provide automated remote software deployment, background daemon push-installation, or remote script execution across enterprise fleets.

- Automated Host Remediation: The platform deliberately avoids executing automated corrective configuration changes or remote remediation commands on monitored endpoints, maintaining an informative, read-only paradigm to eliminate the risk of inadvertent operational outages.

- Guaranteed Concurrent Deduplication: Event deduplication is implemented as an application-level sequential check; it does not provide database-level concurrency guarantees against simultaneous parallel uploads of identical reports.

- Autonomous Diagnostic Guarantees: Artificial intelligence outputs are structured recommendations intended to assist human engineers during investigation; they do not represent infallible or autonomous root-cause determinations.

- Direct Datacenter Facilities Management: Electrical power distribution unit (PDU) actuation and environmental HVAC monitoring remain strictly external to the platform.

1.5 Project Methodology Overview

The engineering methodology employed throughout the EIMS project followed a structured, iterative systems engineering lifecycle. The workflow transitioned through nine sequential phases:

1. Requirements Analysis: Identification of operational pain points, formalization of functional and non-functional requirements, and definition of user personas.

2. Architectural Design: Formal specification of the hybrid modular monolith architecture, database schemas, C4 container boundaries, and RESTful API contracts.

3. Backend and Persistence Implementation: Development of the core FastAPI service, SQLAlchemy relational models, and Alembic database migration scripts.

4. Dashboard Development: Implementation of responsive web interfaces using Next.js, React component hierarchies, Tailwind CSS design tokens, and TypeScript type contracts.

5. Collector Development: Construction of the portable USB Auditor with embedded CPython 3.14.3 amd64 runtimes, native Windows management queries, and automated builder packaging pipelines.

6. Pipeline Integration: Integration of offline JSON multipart ingestion routes, application-level SHA-256 deduplication logic, severity sorting queues, and the RAG-based AI log analysis service.

7. Automated Verification: Testing spanning unit test suites, integration test suites, and frontend static type and production build checks.

8. Field Validation: Deployment and execution of the portable auditor on authorized enterprise test endpoints and production-representative Windows Server hosts.

9. Hardening and Evidence Freeze: Remediation of edge-case bugs, enforcement of strict data anonymization, verification of the downstream data chain, and formal freezing of the graduation baseline.

1.6 Expected Benefits

The implementation of EIMS delivers substantial qualitative and operational benefits to enterprise IT infrastructure operations:

- Centralized Operational Visibility: Unifies previously segregated hardware inventory, operating system attributes, compliance baselines, and event logs into a consolidated, queryable repository.

- Structured Support for Disconnected and Air-Gapped Workflows: Establishes a repeatable, safe methodology for auditing isolated, high-security, or unmanaged Windows endpoints using non-intrusive portable media, eliminating the requirement for complex network routing or risky firewall exceptions.

- Deterministic Evidence Organization: Replaces manual spreadsheet records and unindexed text files with a structured relational schema that links assets, telemetry snapshots, and event records.

- Less Manual Effort to Connect Related Records: Eliminates the necessity for operators to manually aggregate data across multiple local administration consoles, automatically associating incoming operational events with asset identity and configuration state.

- Verifiable Audit Provenance: Ensures that every persisted event and analytical diagnostic finding maintains explicit traceability back to its originating host, collection timestamp, and ingestion transaction.

- Cognitive Workload Reduction via AI-Assisted Prioritization: Mitigates diagnostic fatigue by filtering informational noise, isolating high-severity operational anomalies, and providing contextual remediation references to assist human troubleshooting.

- Transparent Security Posture Auditing: Provides consolidated visibility into vital endpoint protection mechanisms, such as BitLocker encryption, firewall enforcement, and antimalware status, supporting organizational compliance objectives.

1.7 Chapter Summary

This introductory chapter established the operational context, problem domain, objectives, scope, methodology, and expected benefits of the Enterprise Infrastructure Management System (EIMS). EIMS addresses the fundamental friction of managing fragmented infrastructure information and investigating operational incidents across connected and disconnected enterprise endpoints. The subsequent chapters present the complete engineering realization of this system. Chapter 2 details the foundational technical concepts, architectural principles, and related engineering technologies underpinning the platform. Chapter 3 explains the system requirements, design, and implementation of the EIMS backend, portable collector, ingestion pipeline, AI analyzer, and operational dashboard. System validation results and empirical field evidence are subsequently presented in Chapter 4.

Source docs/report/CHAPTER_2_CONCEPTS_AND_RELATED_TECHNOLOGIES.md

Chapter 2

Concepts and Related Technologies

Current graduation revision and evidence boundary

Current source audit: 1 November 2026 — 2a0d9031297ac1063a0f327a8a76c455bb5da315 (PR #10). Next.js 16.3.0 / React 19.2.8. Engineering CI: 391 Python tests (299 Analyzer hermetic + 46 synthetic PostgreSQL + 46 USB isolated), run 41230498604, not project-wide certification. DEV 12 DRAFT, HELDOUT 5 reserved, claims 61, genuine human reviews 0. Diagnostic accuracy NOT_MEASURED; secure HTTPS browser BLOCKED_NOT_TESTED; current USB/production acceptance NOT_TESTED. No main merge.

The verified dashboard dependency versions are Next.js 16.3.0 and React 19.2.8 (clients/dashboard/package.json and lockfile). Embeddings/RAG described below are architectural concepts and legacy capabilities, not the mechanism measured by the frozen M0/M1/Full experiments. M0 uses existing rule logic; M1 adds local vendor knowledge; Full executes actual route synthesis, evidence and in-memory History reconstruction with network/vector writes disabled. M2/RAG and M3/LLM remain NOT_RUN. Citation coordinate validity and semantic support are distinct; only genuine human judgments can establish the latter.

Historical prototype material (retained for provenance)

The purpose of this chapter is to articulate the foundational engineering principles, theoretical concepts, and underlying software technologies required to understand the architecture, data processing models, and operational mechanisms of the Enterprise Infrastructure Management System (EIMS). As an engineering foundation, this chapter delineates generic industry principles and established technologies, setting the technical baseline prior to the detailed presentation of the EIMS implementation in Chapter 3.

2.1 Infrastructure Asset Management

Infrastructure Asset Management (IAM) in enterprise computing constitutes the systematic process of cataloging, monitoring, and governing physical and virtual compute resources throughout their operational lifecycles [1]. An effective asset management framework provides accurate answers regarding what equipment exists within the organization, where it is logically and physically located, how it is configured, and what security posture it maintains.

At the core of asset management is the concept of asset identity. In distributed computing environments, assigning and maintaining immutable identity across physical and virtual compute instances is challenging. Ephemeral attributes such as assigned Internet Protocol (IP) addresses, dynamic hostnames, and localized machine aliases frequently change as systems migrate across network subnets or undergo administrative reconfiguration. Consequently, practical asset management architectures rely on composite cryptographic fingerprints derived from immutable hardware attributes. These attributes typically incorporate Motherboard Serial Numbers, Central Processing Unit (CPU) Identification strings, and Media Access Control (MAC) addresses of physical network interfaces.

Beyond identity, asset management models maintain deep operational metadata across multiple dimensions:

- Hardware Profile: Microprocessor architecture, physical core topology, installed Random Access Memory (RAM) module serials, and physical storage controller disk configurations.

- Operating System and Kernel State: Distribution flavor, kernel build number, patch revisions, and system uptime.

- Network Topology: Primary and secondary interface bindings, subnet masks, default gateways, and Domain Name System (DNS) configurations.

- Security and Compliance Posture: Baseline hardening state, disk encryption presence, host-based firewall configurations, and local endpoint protection status.

A centralized asset registry acts as the central relational anchor across an enterprise. By maintaining unique entity records in a persistent datastore, secondary operational streams—such as time-series performance metrics, security audit logs, diagnostic reports, and operational failure events—can be referentially linked to a specific physical or virtual machine. This referential linkage enables historical tracking, configuration drift analysis, and retrospective root-cause investigations.

2.2 Endpoint Evidence Collection

Endpoint evidence collection denotes the procedural and programmatic acquisition of configuration states, hardware inventories, operational metrics, and event histories directly from host operating systems. In enterprise administration, collection methodologies are broadly bifurcated into two primary paradigms: connected agent-based collection and disconnected portable collection.

Connected collection relies on persistent background services (daemons or Windows services) deployed across managed endpoints. These daemons continuously establish outbound network sockets (such as HTTPS WebSockets or gRPC streams) back to a centralized management collector. While connected architectures provide continuous, near-real-time visibility, they introduce significant operational trade-offs:

- They require continuous network connectivity and liberal firewall routing rules across internal network segments.

- They incur continuous computational overhead (CPU cycles, memory allocation, and thread contention) on monitored systems.

- They expand the attack surface of the endpoint by running privileged listener processes that require perpetual patch management.

Conversely, disconnected or portable collection employs an on-demand, episodic operational paradigm. Under this model, an auditor utility is dispatched to the target endpoint via portable physical media (such as universal serial bus storage devices) or temporary network shares. The collection utility executes locally, interrogates the operating system through native administrative interfaces, structures the gathered telemetry into a standardized, human- and machine-readable data format (such as JSON), writes the artifact directly to the local portable medium, and terminates immediately.

Crucial to endpoint collection is the principle of read-only, non-destructive execution. In enterprise and production environments, an auditing collector must strictly observe and record system state without executing mutating commands, altering registry settings, applying unauthorized patches, or disrupting

running services. Furthermore, in disconnected environments, the local JSON artifact generated directly on the target host serves as the local evidence source, guaranteeing data integrity prior to downstream ingestion into centralized datastores.

2.3 Windows Management Technologies

In environments dominated by the Microsoft Windows operating system family, programmatic interrogation of hardware and system state relies on Windows Management Instrumentation (WMI) and the Common Information Model (CIM) [2].

WMI represents Microsoft's native implementation of Web-Based Enterprise Management (WBEM) and CIM, an open industry standard developed by the Distributed Management Task Force (DMTF). WMI provides an object-oriented, scriptable infrastructure for querying operating system internals, hardware enumerations, device drivers, and service configurations:

- WMI Architecture: WMI functions as an abstraction layer between administrative management tools and underlying operating system components. It operates via the WMI Core (WmiPrvSE.exe provider hosts), a Component Object Model (COM) interface, and a structured schema repository (the CIM repository).

- CIM Classes and Namespaces: WMI structures data into hierarchical namespaces (such as root\cimv2 and root\standardcimv2). Inside these namespaces, standardized CIM classes encapsulate hardware and software attributes:

- Win32_OperatingSystem: Exposes operating system versioning, kernel build architectures, total physical memory, and system drive details.

- Win32_ComputerSystem: Provides domain membership, physical chassis vendor, and system manufacturer identifiers.

- Win32_Processor: Details processor SKU, physical socket count, logical core topology, and hardware identification.

- Win32_NetworkAdapterConfiguration: Enumerates active physical network interface controllers (NICs), MAC addresses, IP bindings, and DHCP lease data.

- Win32_DiskDrive and Win32_LogicalDisk: Provide physical disk topology, serial numbers, partition layouts, and file system volume metrics.

In modern Windows environments (specifically Windows PowerShell 3.0 and newer), native CIM cmdlets (such as Get-CimInstance) have largely superseded legacy WMI cmdlets (Get-WmiObject). CIM cmdlets operate over the standardized WS-Management (WS-Man) protocol and native Windows APIs, delivering improved execution performance, thread-safe asynchronous invocation, and robust memory management when retrieving complex system structures.

Beyond hardware inventory, Windows administrative queries leverage specialized platform management cmdlets to evaluate endpoint security postures:

- Get-Service: Enumerates local Windows service configurations and runtime states.

- Get-BitLockerVolume: Interrogates volume-level BitLocker drive encryption parameters, including volume protection status, encryption percentage, encryption algorithm, and active key protector metadata.

- Get-MpComputerStatus: Retrieves Windows Defender antimalware status, real-time protection flags, engine versions, and signature update timestamps.

- Get-NetFirewallProfile: Queries Windows Advanced Firewall policy states across Domain, Private, and Public network profiles.

In portable execution environments where external C++ compilation tools or Python COM bindings (pywin32) cannot be guaranteed on target hosts, executing these native cmdlets via bounded child processes represents a reliable, zero-dependency pattern for system interrogation.

2.4 Windows Event Logging

The Microsoft Windows Event Log infrastructure provides a centralized, standardized subsystem for recording hardware, operating system, application, and security occurrences [3]. Originating with the Windows Vista and Windows Server 2008 architectures, Windows Event Logging superseded the legacy Event Logging format, replacing flat textual logs with structured, schema-validated binary XML (.evtx) files stored within %SystemRoot%\System32\Winevt\Logs\.

Event Log Structural Hierarchy

A native Windows Event comprises six essential structural elements:

1. Channel: The delivery pipeline and storage container for the event. Standard Windows deployments define four primary administrative channels: Application, System, Security, and Setup, augmented by hundreds of granular diagnostic and operational channels.

2. Provider (Source): The software entity, device driver, or operating system component that publishes the event (e.g., Service Control Manager, Microsoft-Windows-Kernel-Power, EventLog).

3. Event Identifier (Event ID): A 16-bit or 32-bit numeric integer uniquely identifying the specific occurrence type within the scope of the publishing provider.

4. Severity Level: The operational criticality assigned to the event, categorized into standardized hierarchical levels:

- Critical (Level 1): Indicates an unrecoverable failure that caused application or system termination.

- Error (Level 2): Denotes a significant operational failure that impacts service delivery but allows partial system execution.

- Warning (Level 3): Flags potential issues or unexpected states that may foreshadow future operational degradation.

- Information (Level 4): Confirms normal operational milestones (e.g., service starts, successful driver loads).

- Verbose (Level 5): Fine-grained progress and debugging tracing.

5. Timestamp (occurrence_time): High-precision UTC timestamp recording the exact instance the operating system kernel generated the event.

6. Event Record Identifier (record_id): A monotonically increasing 64-bit integer assigned by the local event log service, uniquely indexing the event sequentially within that specific channel on that local machine.

Operational Channel Selection

For operational troubleshooting and infrastructure management, the System and Application channels represent the most informative telemetry streams. The System channel records core operating system events, hardware controller failures, service start/stop events, storage subsystem degradation, and driver

crashes. The Application channel records third-party enterprise software errors, database process terminations, and software dependency failures.

Conversely, the Security channel is dedicated strictly to security auditing (e.g., logon/logoff monitoring, user rights assignment, and object access). The portable graduation workflow intentionally scopes Windows Event Log collection to the System and Application channels, focusing diagnostic evidence acquisition directly on core operating system reliability, hardware health, and enterprise software execution.

2.5 Evidence Provenance and Deduplication

In distributed systems engineering, data provenance denotes the end-to-end documentation of data origin, origin, transformations, and handling history [4]. When operational logs and diagnostic reports are gathered across distributed hosts and ingested into a central platform, establishing traceable event and asset metadata is critical to validating the integrity of downstream analytical findings.

Data provenance in evidence management requires explicit referential linkage across three architectural tiers:

1. Asset Linkage: Every ingested artifact must reference a verified canonical asset identifier (asset_id), binding operational records directly to a physical or virtual machine entity.

2. Source Classification: Ingested payloads must carry explicit categorical provenance markers (e.g., source_type = "USB_OFFLINE_COLLECTION"), formally differentiating data gathered through air-gapped portable audits from telemetry streamed via connected network daemons.

3. Event-Source Linkage: Downstream analytical findings, automated diagnostic summaries, and operator annotations must store direct foreign references (event_source_id) pointing back to the specific originating event log entity, preventing orphan conclusions.

Deduplication Architecture: Application-Level vs. Database Constraints

A major hazard in periodic offline evidence collection is the redundant ingestion of identical operational records. If an operator conducts multiple USB audits across overlapping time windows on the same server, subsequent uploads will transmit previously ingested events.

To neutralize record duplication, systems implement deterministic cryptographic hashing. An event's immutable identity is computed by concatenating its canonical parameters:

DedupKey = SHA-256(asset_id | channel | provider | record_id | occurrence_time)

Because the record_id is monotonically assigned per channel on the target host and the occurrence_time is locked to the event's recorded timestamp, this hash is deterministic and collision-resistant across multiple extractions of the same physical event.

In software architecture, deduplication can be enforced at the database level or the application level:

- Database-Level Enforcement: Implemented via relational UNIQUE constraints or composite primary keys. The database engine guarantees transactional uniqueness and prevents concurrent race conditions across simultaneous parallel transactions, but incurs significant index leaf insertion overhead and requires handling database-specific transaction abort exceptions.

- Application-Level Sequential Deduplication: Implemented within application services prior to persistence execution. The service queries existing deduplication keys for the target asset, filters duplicate records in memory, and persists only novel entities.

While application-level sequential deduplication effectively protects against duplicate record insertion during standard, sequential report imports, it does not provide database-level concurrency guarantees. If two identical reports are submitted simultaneously across concurrent threads, race conditions can occur. This distinction is an important architectural boundary of the system.

2.6 Event Prioritization

Enterprise Windows endpoints routinely produce hundreds or thousands of event records within a standard 24-hour monitoring window. Transmitting, indexing, and executing artificial intelligence analysis across every individual informational event is computationally inefficient and operationally counterproductive.

To optimize computational efficiency and focus engineering attention, event processing architectures apply deterministic severity prioritization. Events are stratified based on their standard Windows severity classifications:

Critical (Level 1) > Error (Level 2) > Warning (Level 3) > Information (Level 4)

Operational prioritization serves two essential functions:

1. Resource Bounding: Downstream natural language processing (NLP) and large language model (LLM) analytical engines incur substantial computational latencies and token processing costs. Imposing a strict upper bound (such as a maximum of 10 prioritized events per batch import) guarantees that system resources are dedicated exclusively to critical errors and warnings while safeguarding the ingestion pipeline against backpressure.

2. Diagnostic Clarity: Systems administrators resolving an infrastructure incident require clear summaries of errors and warnings rather than being overwhelmed by hundreds of routine informational notifications.

When sorting candidate events within identical severity levels, deterministic tie-breaking rules (such as chronological recency or stable input order) must be enforced to ensure deterministic, reproducible ingestion behavior.

2.7 AI-assisted Log Analysis

Artificial intelligence and natural language processing techniques have increasingly transitioned from academic research into applied infrastructure engineering. When applied to system administration, AI models serve as intelligent investigation aids, accelerating the triage of complex operational failures without replacing human operational authority [5].

Semantic Vector Embeddings

Traditional log analysis relied predominantly on regular expressions (regex) and keyword substring matching (e.g., searching for "failed" or "timeout"). However, raw event log messages exhibit high syntactic variance: different software providers and driver versions articulate identical underlying failure modes using completely distinct terminology.

To overcome lexical brittleness, modern systems leverage semantic vector embeddings. An embedding model, such as FastEmbed with BAAI/bge-small-en-v1.5 primary and SentenceTransformer all-MiniLM-L6-v2 [6] fallback, maps textual log descriptions into a dense, continuous vector space (specifically a 384-

dimensional space). In this embedding space, semantically similar sentences are positioned in close spatial proximity, regardless of whether they share exact vocabulary. The mathematical similarity between two text embeddings u and v is evaluated using Cosine Distance:

CosineDistance(u, v) = 1 - (u · v) / (||u||_2 * ||v||_2)

Where a Cosine Distance approaching 0 denotes near-identical semantic intent.

Retrieval-Augmented Generation (RAG) Architecture

While Large Language Models exhibit impressive conversational reasoning, deploying raw LLMs directly against enterprise logs introduces critical failure modes:

- Hallucination: General-purpose models frequently invent non-existent command-line switches, erroneous registry keys, or fictional technical manuals when attempting to resolve obscure operating system errors.

- Absence of Domain Context: Pre-trained models lack awareness of internal enterprise knowledge bases, historical incident post-mortems, and specific infrastructure architectures.

RAG supplies retrieved context, but generated advice still requires verification [7]. In a RAG pipeline:

1. An incoming, prioritized operational event is parsed and vectorized into an embedding.

2. A high-performance vector database executes an approximate nearest neighbor (ANN) or exact k-nearest neighbor search to retrieve relevant, pre-curated diagnostic solutions and historical incident resolutions matching the event.

3. The retrieved reference snippets are injected into a structured prompt alongside the raw event metadata.

4. The language model synthesizes a concise, context-grounded diagnostic summary and remediation plan guided by the retrieved reference material.

Crucially, in operational infrastructure management, AI-generated findings must be positioned strictly as diagnostic triage assistance rather than autonomous root-cause determinations.

2.8 Relational and Semi-Structured Data Storage

Persistent data management in enterprise infrastructure platforms necessitates balancing rigorous transactional integrity against the polymorphic flexibility required to store diverse hardware and telemetry structures.

PostgreSQL Relational Datastore

PostgreSQL is an advanced, open-source object-relational database management system known for reliability, feature robustness, and strict adherence to ANSI SQL standards [8]. PostgreSQL provides full ACID (Atomicity, Consistency, Isolation, Durability) guarantees, ensuring that multi-table transactions—such as registering an asset, inserting hardware components, and logging an audit transaction—either commit entirely or roll back safely without corrupting database integrity.

Handling Polymorphic Telemetry via JSONB

While core enterprise entities (such as assets, users, and audit records) map naturally into normalized relational schemas, diagnostic telemetry exhibits extreme structural heterogeneity. A hardware telemetry payload emitted by a Dell PowerEdge server contains storage controller metrics that bear no structural resemblance to an operating system event emitted by a Windows 11 workstation.

Rather than forcing heterogeneous data into fragmented, sparse relational tables or abandoning relational integrity for schema-less NoSQL databases, modern architectures utilize PostgreSQL's native JSONB (binary JSON) data type. JSONB stores decomposed binary representations of semi-structured JSON documents, supporting relational indexing via Generalized Inverted Indexes (GIN):

- GIN Indexing: GIN indexes construct internal B-tree structures over all keys and elements within a JSONB document. Using specialized operator classes (such as jsonb_path_ops), the database engine can execute sub-millisecond containment queries (@>) or key-existence queries without sequentially scanning entire relational tables.

Vector Persistence via pgvector

To support artificial intelligence and semantic search within the relational boundary, the open-source PostgreSQL extension pgvector introduces native vector data types and specialized mathematical index structures [9]. pgvector enables database engines to store dense vector representations (e.g., Vector(384)) directly alongside relational attributes, supporting exact and approximate nearest neighbor queries using cosine distance (<=>), L2 Euclidean distance (<->), or inner product (<#>) operators natively within standard SQL statements. In this architecture, pgvector functions as a specialized indexing and distance-calculating persistence layer, while embedding models and language processors execute within separate application runtimes.

2.9 Redis and Asynchronous Processing

In high-throughput telemetry platforms, direct synchronous database writes create severe performance bottlenecks. If hundreds of endpoints submit diagnostic reports simultaneously, or if long-running analytical operations (such as OCR extraction or LLM embedding generation) are executed synchronously within HTTP request lifecycles, backend web workers quickly exhaust their connection pools, leading to thread starvation and elevated request latency.

To decouple ingestion from persistence and processing, architectures integrate Redis [10]. Redis is an open-source, in-memory key-value data structure store operating with sub-millisecond retrieval latencies. In modern distributed platforms, Redis fulfills three vital architectural functions:

1. Asynchronous Ingestion Broker: Incoming diagnostic payloads are rapidly accepted by API gateways, validated, and pushed onto Redis message queues or Redis Streams. The API gateway immediately returns an HTTP 202 Accepted response to the client, while dedicated background worker processes consume payloads from the queue and execute batch database operations out of band.

2. Distributed Caching: Frequently accessed, computationally expensive database queries—such as asset compliance distributions and global infrastructure counts—are cached in Redis with defined Time-to-Live (TTL) expiration windows.

3. Session and State Tracking: Transient tokens, rate-limiting quotas, and active user session states are maintained in Redis to enable rapid validation without burdening primary relational databases.

2.10 Object Storage and Optical Character Recognition (OCR)

In addition to structured telemetry and relational metadata, enterprise asset onboarding frequently involves unstructured binary files. Specifically, physical server deployments produce paper hardware manifests, shipping invoices, and physical chassis specification stickers.

To persist binary imagery securely and scalably, modern architectures utilize S3-compatible object storage engines, such as MinIO [11]. Object storage abstracts physical storage drives into logical buckets containing immutable binary blobs accessible via standard RESTful HTTP/S3 APIs. Rather than storing large binary blobs (BLOBs) inside relational database tables—which causes severe database bloat and degrades database backup performance—the system stores the raw image in MinIO and maintains the corresponding Uniform Resource Identifier (URI) within PostgreSQL.

To extract actionable asset metadata from physical server chassis stickers or shipping labels, systems deploy Optical Character Recognition (OCR) pipelines [12]. An OCR engine extracts alphanumeric text strings from image binaries, which are subsequently parsed via regular expression heuristics to extract vendor part numbers, serial codes, and hardware specifications. Within EIMS, OCR serves as a specialized supporting capability to accelerate manual hardware entry rather than acting as the primary telemetry collection path.

2.11 Backend API Framework

The backend routing and business logic of modern infrastructure platforms require high concurrency, strong typing guarantees, and standardized interface specifications.

FastAPI is a modern, high-performance web framework for building APIs with Python 3.10+ based on standard Python type hints [13]. FastAPI is constructed upon two foundational libraries:

- Starlette: Provides lightweight, high-performance Asynchronous Server Gateway Interface (ASGI) routing, WebSocket support, and background task management.

- Pydantic: Delivers deep data validation, type enforcement, and serialization using Python type annotations. Incoming JSON payloads are automatically deserialized into strongly typed schema models, rejecting malformed requests with detailed HTTP 422 Unprocessable Entity error envelopes before reaching application business logic.

- OpenAPI Compilation: FastAPI automatically inspects endpoint definitions, type models, and docstrings, compiling standard OpenAPI 3.1 and JSON Schema definitions. This enables the automatic generation of interactive API documentation (Swagger UI and ReDoc) and guarantees synchronization between backend code and client interface contracts.

2.12 Frontend Architecture

Operational infrastructure dashboards require modern, responsive web interfaces capable of presenting dense, complex telemetry clearly to system operators without introducing noticeable interface latency.

Next.js is a production-tested React framework providing hybrid rendering architectures, file-system routing, and built-in optimization pipelines [14]. In complex administrative dashboards, frontend architecture balances two rendering models:

- Server-Side Rendering (SSR): Components are pre-rendered into static HTML on the server runtime, accelerating First Contentful Paint (FCP) and ensuring that initial page loads display complete layouts immediately.

- Client-Side Rendering (CSR): Interactive components, such as dynamic telemetry charts, modal inspection dialogues, and keyboard-navigated search palettes, execute dynamically within the client browser DOM using React state hooks (useState, useEffect).

Hydration as an Architectural Consideration

A critical architectural consideration in hybrid Next.js/React applications is the hydration process. Hydration occurs when the client-side JavaScript engine reads the pre-rendered HTML DOM emitted by the server and attaches client-side event listeners and state management trees. If the server-rendered HTML diverges from the initial client-rendered DOM—for instance, if responsive sidebar navigation components calculate browser viewport dimensions (window.innerWidth) that do not exist during server execution—React triggers a hydration mismatch warning, resulting in UI flickering or layout shifts. Managing hydration boundaries through defensive layout mounting and client-only state wrapping is essential for delivering reliable web interfaces.

2.13 Authentication and Security Boundary

Enterprise infrastructure systems manage privileged access to hardware assets, requiring well-defined authentication and authorization boundaries [15].

Authentication Modes: Demo vs. Secure Architecture

Software engineering platforms developed through iterative milestones often incorporate configurable authentication operational modes:

- Demo Mode: Designed for rapid local evaluation, developer testing, and automated integration suites in isolated sandbox environments. In Demo Mode, administrative endpoints provide trusted bypass mechanisms, returning standardized mock identities (such as a default demo operator) without requiring external authentication tokens.

- Secure Mode: Enforces strict cryptographic identity verification. All protected API interfaces require valid, signed JSON Web Tokens (JWT) conforming to RFC 7519 [16]. Requests lacking valid authorization headers or carrying expired tokens are rejected immediately with HTTP 401 Unauthorized errors.

Server-Side Administrative Boundary

In secure enterprise architectures, privileged administrative credentials—such as administrative write tokens or master database keys—must reside strictly within backend server environments. Client-side web dashboards executing in user browsers must never embed hardcoded administrative tokens within compiled JavaScript bundles. Write operations, such as asset lifecycle state mutations or system configuration updates, must be authorized exclusively through validated session tokens verified against server-side authorization middleware.

2.14 Observability Technologies

Operational stability in multi-service platforms depends on comprehensive observability, traditionally categorized into metrics, logs, and traces [17].

- Prometheus: An open-source systems monitoring and alerting toolkit operating on a pull-based metrics scraping architecture [18]. Prometheus collects numerical time-series metrics (such as CPU consumption, memory saturation, HTTP request latencies, and active database connections) from application endpoints exposing standardized /metrics interfaces.

- Grafana: A multi-platform visualization suite that queries Prometheus data stores, rendering interactive, real-time dashboards displaying platform health, API throughput, and system resource saturation.

- Health Check Endpoints: Standardized REST endpoints (such as GET /api/v1/health) that execute non-destructive diagnostic probes against core subsystem dependencies (PostgreSQL database connectivity, Redis broker responsiveness, and MinIO storage availability), returning structured health summaries to monitoring agents and container orchestrators.

2.15 Related Engineering Approaches

To contextualize the architectural positioning of EIMS, it is instructive to examine existing commercial and open-source engineering paradigms across related IT infrastructure domains. The table below presents a neutral, descriptive comparison across foundational operational dimensions.

Paradigm / Category	Exemplary Representative Systems	Primary Operational Domain	Typical Telemetry Ingestion Model	Offline / Portable Media Collection
Infrastructure Monitoring Systems	Nagios, Zabbix, Datadog [17]	Real-time host availability, hardware metrics, and uptime alerting.	Active network polling (SNMP/ICMP) or streaming agent daemons.	Typically unsupported; architectures assume continuous network reachability.
IT Asset Management (ITAM / CMDB)	ServiceNow CMDB, Snipe-IT [1]	Hardware procurement, lifecycle accounting, depreciation, and ownership.	Periodic inventory imports, manual database forms, or discovery scans.	Primarily text/CSV imports; lacks native operating system event log triage.
Log Management & SIEM	Splunk, Elastic SIEM [5]	Enterprise security auditing, regulatory compliance, and threat correlation.	Continuous high-velocity log forwarders (syslog, agent beats).	Designed for enterprise network streams; requires substantial cluster infrastructure.
Endpoint Security Agents	OSquery, Wazuh [15]	Host-based intrusion detection, compliance queries, and file integrity monitoring.	Persistent background daemons executing scheduled local checks.	Requires persistent daemon installation and ongoing network connectivity.
Offline Diagnostic Collectors	Microsoft Sysinternals, Dell SupportAssist [2]	Ad-hoc local troubleshooting and component hardware diagnostics.	Local interactive execution producing standalone text/HTML reports.	Standalone local report generation; traditionally unintegrated with central AI triage.

As illustrated by this descriptive comparison, EIMS focuses on collecting and linking evidence from disconnected Windows hosts. This project did not benchmark EIMS against commercial SIEM products. Rather than functioning as a fleet-scale SIEM or a financial asset accounting ledger, EIMS unifies infrastructure asset registry tracking, air-gapped portable evidence acquisition, centralized relational ingestion, and AI-assisted operational event triage within a lightweight, coherent platform architecture.

2.16 Chapter Summary

This chapter presented the theoretical concepts, foundational standards, and technical architectures underlying EIMS. The discussion explored the principles of infrastructure asset management and cryptographic hardware identity, the trade-offs between connected and portable endpoint collection, the structure of Windows WMI/CIM and binary Event Logs, the mathematics of semantic embeddings and RAG architectures, and the persistent capabilities of PostgreSQL, JSONB, and pgvector. Furthermore, the

operational roles of Redis, MinIO, FastAPI, Next.js, and observability stacks were systematically defined. With these foundational engineering concepts established, Chapter 3 details the concrete system design, software architecture, and implementation details of the EIMS platform.

Source docs/report/CHAPTER_3_SYSTEM_DESIGN_AND_IMPLEMENTATION.md

Chapter 3

System Design and Implementation

Current graduation revision and evidence boundary

Current source audit: 1 November 2026 — 2a0d9031297ac1063a0f327a8a76c455bb5da315 (PR #10). Next.js 16.3.0 / React 19.2.8. Engineering CI: 391 Python tests (299 Analyzer hermetic + 46 synthetic PostgreSQL + 46 USB isolated), run 41230498604, not project-wide certification. DEV 12 DRAFT, HELDOUT 5 reserved, claims 61, genuine human reviews 0. Diagnostic accuracy NOT_MEASURED; secure HTTPS browser BLOCKED_NOT_TESTED; current USB/production acceptance NOT_TESTED. No main merge.

Current core workflow: authenticated browser → Next.js BFF (HttpOnly/Secure cookie) → FastAPI Analyzer → normalized source records/vendor rules/local knowledge → structured investigation → user-owned PostgreSQL History → saved source-context viewer. Server-side revocation and current user ownership apply to protected requests. No failure observed does not imply global health; CTK technical failure does not prove job failure or physical cause; Event 1129 identifies a DC-connectivity symptom. Fortinet configuration event identity alone is not failure. Unknown completion is not resolved by an unrelated session. Customer-scoped forms, general-user shared organizational APIs and authenticated WebSocket streaming are not certified live workflows. The wider platform design below is implemented/historical scope, not proof every path was accepted on this revision.

Historical prototype material (retained for provenance)

This chapter explains the system requirements, design, and implementation of the Enterprise Infrastructure Management System (EIMS). It details the functional and non-functional requirements, presents the overall system architecture with structural diagrams, specifies the relational and vector data models, documents the portable collection utility, details the ingestion and AI-assisted analysis pipelines, and formalizes the validation and security governance frameworks.

3.1 Requirement Analysis

The design of EIMS originated from a formal requirements engineering phase conducted to address enterprise infrastructure visibility and diagnostic bottlenecks. System requirements were formalized within the EIMS Product Requirements Document (EIMS-PRD-001) as canonical architectural specifications.

3.1.1 Operational Personas

EIMS defines four distinct operational personas reflecting enterprise organizational boundaries:

1. System Administrator (Operator): Oversees enterprise infrastructure availability, registers managed compute endpoints, executes or reviews offline evidence imports, and configures platform settings.

2. Security & Compliance Auditor: Inspects endpoint compliance baselines, audits disk encryption postures and antimalware states, and reviews system mutation audit logs.

3. Hardware Field Technician: Deploys physical server infrastructure, executes portable USB audits on air-gapped or disconnected machines, and uploads hardware manifest documentation.

4. Discovery Agent / Collector (Service Account): Automated execution utility deployed on target endpoints, responsible for gathering system configurations, interrogating operating system events, and structuring JSON evidence payloads.

3.1.2 Canonical Architecture & Target Requirements (PRD Baseline)

The platform requirements originated in the Product Requirements Document (EIMS-PRD-001) as long-term architectural specifications and target baselines:

Canonical Functional Requirements

- REQ-REG-01 (Canonical Entity Indexing): The system must assign a universally unique identifier (UUIDv4) to every verified infrastructure asset, persisting canonical properties (hostname, assigned IP address, operating system kernel, lifecycle state) in normalized relational tables.

- REQ-REG-02 (Telemetry State Deduplication): Ingestion pipelines must identify existing asset records via composite cryptographic fingerprints (derived from hardware serials and MAC addresses) to execute relational upsert operations, preventing redundant entity creation.

- REQ-REG-03 (Lifecycle State Machine Enforcement): The platform must constrain asset state transitions strictly to approved operational lifecycle stages: Discovered, PendingAudit, Compliant, NonCompliant, Quarantined, and Decommissioned.

- REQ-DISC-01 (Agent Autonomous Enrollment): Endpoint collectors must register unindexed endpoints by generating a unique cryptographic fingerprint derived from immutable hardware attributes.

- REQ-DISC-02 (Telemetry Payload Streaming): Connected agent daemons must collect operating system health metrics and stream validated JSON payloads via HTTPS.

- REQ-DISC-03 (Asynchronous Queue Buffering): The API gateway must deserialize incoming payloads, validate structure against domain schemas, and enqueue raw telemetry into in-memory broker queues for asynchronous processing.

- REQ-DISC-04 (Offline Exception Recovery): Endpoint collection utilities must support bounded local file buffering when network connectivity to the central platform is unavailable.

- REQ-LOG-01 (Windows Event Log Ingestion): The collection pipeline must extract Windows Event Logs, filtering diagnostic noise and streaming operational events to the central platform.

- REQ-LOG-02 (Anomaly & IoC Rule Evaluation): Background ingestion engines must evaluate operational event streams to identify operational failures and security exceptions.

- REQ-LOG-03 (Security Exception Alerting): Critical operational failures and security exceptions must be recorded within the audit journal and dispatched to administrative interfaces.

- REQ-COMP-01 (Automated Baseline Evaluation): The system must compare active endpoint configurations against established baseline rules (antimalware state, firewall enforcement, BitLocker encryption).

- REQ-COMP-02 (Dynamic Compliance Calculation): The engine must calculate a composite numeric compliance score ranging from 0 to 100 for every monitored endpoint.

- REQ-COMP-03 (Automated Quarantine Threshold): If an asset's computed compliance score drops beneath the acceptable threshold (score < 70), the entity state must transition to Quarantined.

- REQ-OCR-01 (Multipart Manifest Ingestion): The API gateway must expose dedicated multipart endpoints accepting physical hardware purchase documentation, shipping labels, and specification faceplate imagery.

- REQ-OCR-02 (Object Storage Archival): Uploaded binary documents must be archived into local S3-compatible object storage buckets, storing the object URI in relational database tables.

- REQ-OCR-03 (Autonomous Hardware Parsing): Asynchronous background workers must extract text strings from stored image binaries to populate hardware inventory records.

- REQ-UI-01 (Real-Time Observability Portal): The web interface must render a responsive operational dashboard visualizing asset inventory distributions, compliance scores, active OCR tasks, and operational events.

- REQ-UI-02 (Live Telemetry Streaming): Dashboard client sessions must support live metric rendering without requiring manual page reloads.

- REQ-UI-03 (RBAC Enforcement UI): The interface must enforce role-based access control, rendering administrative controls for privileged operators while restricting analytical views for read-only auditors.

3.1.3 Canonical Non-Functional Requirements

- NFR-PERF-01 (API Latency): Synchronous REST API read operations querying the asset registry must achieve response latencies under 50 milliseconds at the 99th percentile (p99) under baseline operational loads.

- NFR-PERF-02 (Ingestion Throughput): The combined gateway and caching tier must reliably absorb burst telemetry payloads without queue backpressure.

- NFR-PERF-03 (OCR Processing Duration): Asynchronous OCR extraction pipelines must complete text parsing and database commits within bounded operational thresholds.

- NFR-REL-01 (Database Recovery): The relational persistence architecture must support rapid database recovery with zero loss of committed transactions (RPO = 0).

- NFR-REL-02 (Agent Resilience): Endpoint collection daemons must handle operating system restarts and process crashes gracefully without requiring manual intervention.

- NFR-SEC-01 (Transport Encryption): All external network communications spanning agents, web clients, and backend gateways must negotiate modern TLS encryption.

- NFR-SEC-02 (Audit Log Immutability): Operational state mutations must be recorded inside an append-only audit log table preventing unauthorized modification or deletion.

- NFR-SEC-03 (Credential Storage): Operator passwords and access tokens must undergo one-way cryptographic hashing before relational persistence.

- NFR-SCALE-01 (Horizontal Stateless Scaling): Backend ingestion gateways and web interface runners must maintain stateless execution contexts to support containerized scaling.

- NFR-SCALE-02 (Telemetry Data Retention): Historical time-series metrics must support chronological table partitioning and long-term archival.

3.1.4 Implemented Graduation System Scope

A crucial distinction must be maintained between the initial target specifications in the canonical PRD and the verified implementation delivered for the cooperative education graduation baseline (v0.3.0 with post-release operational hardening):

1. Streaming Agents vs. Portable Offline Collection: While the canonical PRD specifies autonomous background discovery daemons continuously streaming telemetry via mTLS/WebSockets (REQ-DISC-01, REQ-DISC-02), the graduation implementation prioritizes the zero-dependency Portable USB Auditor (USB_OFFLINE_COLLECTION). This directly resolves the core enterprise operational constraint where persistent third-party daemons are restricted on mission-critical, air-gapped, or segmented endpoints.

2. Event Channel Scope: While the canonical PRD outlines generalized multi-channel event ingestion (REQ-LOG-01), the portable graduation workflow intentionally scopes Windows Event Log collection to the System and Application channels.

3. Automated Enforcement vs. Non-Destructive Posture: While the PRD specifies automated quarantine mechanisms that block management APIs (REQ-COMP-03), the implemented system calculates dynamic compliance scores and reflects quarantined states visually on the dashboard, strictly preserving a non-destructive, read-only operational posture.

4. Infrastructure Architecture vs. Distributed Clustering: While the PRD non-functional requirements define horizontal Kubernetes scaling (NFR-SCALE-01), sub-30-second automated database failover (NFR-REL-01), and sustained 5,000 req/sec ingestion (NFR-PERF-02), the graduation prototype is deployed via single-node container services using Docker Compose with PgBouncer connection pooling. Distributed clustering, Redis Sentinel, and Kubernetes orchestration remain post-graduation architectural roadmaps.

5. Sequential Deduplication & Bounded AI Workload: The graduation release introduces application-level sequential deduplication and bounds AI triage to a maximum of 10 prioritized events per batch import, ensuring predictable resource consumption on single-node deployments.

3.2 Overall System Architecture

EIMS is implemented as a Hybrid Modular Monolith paired with an Asynchronous Ingestion Pipeline. This architectural style was chosen over a distributed microservices pattern to eliminate unnecessary network serialization latency, reduce inter-service failure domains, and maintain transactional database integrity within a cohesive codebase, while isolating high-throughput ingestion workflows via asynchronous messaging.

Major Architectural Components

1. Target Windows Endpoints: Physical servers, virtual machines, or workstations running supported Windows operating systems.

2. Portable USB Auditor: A zero-dependency, self-contained auditing package featuring an embedded CPython 3.14.3 amd64 runtime, executed on target endpoints via batch scripts to generate local JSON reports.

3. FastAPI Core Gateway: High-performance asynchronous Python backend providing RESTful API endpoints, Pydantic request validation, business logic orchestration, and OpenAPI contract publication.

4. PostgreSQL Relational Datastore: Primary persistent ACID datastore equipped with the pgvector extension for semantic vector similarity search and JSONB binary structures for polymorphic telemetry.

5. Redis Cache & Event Broker: In-memory key-value data store used for transient message queuing, rate limiting, and caching computationally intensive query aggregates.

6. PgBouncer Connection Pooler: Lightweight connection pooler managing PostgreSQL connection lifecycle, mitigating database thread exhaustion under high connection concurrency.

7. MinIO Object Storage: S3-compatible local object storage storing binary files, including hardware sticker images and shipping documentation.

8. AI Log Analyzer: Semantic retrieval and diagnostic triage service utilizing FastEmbed (BAAI/bge-small-en-v1.5 primary, with SentenceTransformer all-MiniLM-L6-v2 fallback) and pgvector cosine distance matching to generate contextual mitigation advice.

9. Next.js Operational Dashboard: Responsive web application built with React and TypeScript, providing interactive interfaces for asset management, event investigation, global search, and timeline visualization.

10. Observability Stack: Prometheus metrics scraper and Grafana visualization engine monitoring host and application telemetry via standardized health probe endpoints.

System Architecture Diagram

Source listing mermaid

```
flowchart TB
subgraph Target_Host["Target Windows Endpoint (Connected or Disconnected)"]
WMI[WMI / CIM Subsystem]
EVTX[Windows Event Logs\nSystem & Application]
USB_EXEC["Portable USB Auditor\n(Embedded CPython 3.14.3 + BAT Launcher)"]
WMI --> USB_EXEC
EVTX --> USB_EXEC
JSON_OUT[("Local Evidence JSON Report\nreports/audit_*.json")]
USB_EXEC --> JSON_OUT
end
JSON_OUT -. "Physical Media Transfer or\nLocal API Upload" .-> API_GATEWAY
subgraph EIMS_Core["EIMS Core Platform (Containerized Backend)"]
API_GATEWAY["FastAPI Core Gateway\nREST API / Ingestion Controller"]
subgraph Ingestion_Pipeline["Ingestion & Analysis Pipeline"]
PARSER["JSON Schema Normalizer"]
DEDUP["Application-Level Dedup\nSHA-256 Fingerprinting"]
PRIORITY["Severity Prioritization Queue\nCritical > Error > Warning"]
ANALYZER["AI Log Analyzer\n(FastEmbed BAAI/bge-small-en-v1.5 / all-MiniLM-L6-v2)"]
PARSER --> DEDUP
DEDUP --> PRIORITY
PRIORITY --> ANALYZER
end
API_GATEWAY --> PARSER
end
subgraph Data_Tier["Persistence & Caching Tier"]
PG[("PostgreSQL 16 + pgvector\nAsset Registry & Event Store\nAnalysisHistory & Audit Logs")]
PGBOUNCER["PgBouncer Connection Pooler"]
REDIS[("Redis 7\nCache & Message Broker")]
MINIO[("MinIO S3 Storage\nHardware Manifest Imagery")]
PGBOUNCER --> PG
DEDUP <--> PGBOUNCER
ANALYZER <--> PGBOUNCER
API_GATEWAY <--> PGBOUNCER
API_GATEWAY <--> REDIS
API_GATEWAY <--> MINIO
end
subgraph Presentation_Tier["Presentation Tier"]
DASHBOARD["Next.js Operational Dashboard\nReact / TypeScript / Tailwind CSS"]
```

```
DASHBOARD <--> API_GATEWAY
end
subgraph Observability_Tier["Observability Tier"]
PROM["Prometheus Metrics Scraper"]
GRAFANA["Grafana Dashboards"]
PROM --> API_GATEWAY
GRAFANA --> PROM
end
```

3.3 Asset Registry and Data Model

The EIMS persistent data model balances normalized relational entities with semi-structured JSONB columns.

Core Relational Entities

1. infrastructure_assets: The central entity representing a registered compute endpoint.

- asset_id (UUIDv4, Primary Key): Canonical asset identifier.

- hostname (VARCHAR): Operating system networking hostname.

- canonical_ip (INET): Primary network IP address.

- cryptographic_fingerprint (VARCHAR, Unique): SHA-256 hash of immutable hardware serials and MAC addresses.

- lifecycle_state (VARCHAR): State machine indicator (Discovered, Compliant, NonCompliant, Quarantined, Decommissioned).

- current_compliance_score (SMALLINT): Integer score (0–100) reflecting security posture compliance.

- created_at, updated_at (TIMESTAMPTZ): Temporal tracking timestamps.

2. windows_event_logs: Persists extracted Windows operating system events.

- log_id (UUIDv4, Primary Key): Unique log processing identifier.

- asset_id (UUIDv4, Foreign Key): Reference binding the log to infrastructure_assets.asset_id (ON DELETE CASCADE).

- occurrence_time (TIMESTAMPTZ): Origin timestamp recorded by the Windows operating system kernel.

- event_id (INTEGER): Canonical Windows numeric event code.

- severity_level (VARCHAR): Severity classification (Critical, Error, Warning, Information).

- evtx_metadata (JSONB): Structured dictionary containing the event provider, channel, record identifier, message text, and cryptographic deduplication hash (_reporter_dedup).

3. analysis_history: Stores diagnostic findings, vector search matches, and mitigation plans generated by the AI Analyzer.

- id (INTEGER, Primary Key): Monotonically increasing record ID.

- event_id (VARCHAR): Windows event identifier analyzed.

- provider (VARCHAR): Originating event provider.

- parse_method (VARCHAR): Source provenance classification string (USB_OFFLINE_COLLECTION:USB_OFFLINE_COLLECTION).

- description (VARCHAR): Normalized event textual description.

- ai_summary (VARCHAR): Synthesized diagnostic explanation and remediation plan.

- solution_summary (JSON): Structured mitigation parameters and reference links.

- event_metadata (JSON): Provenance envelope binding the finding to asset_id, event_source_id, channel, and deduplication keys.

- search_results (JSON): Matched knowledge base records and similarity metrics.

- created_at (DATETIME): Ingestion and analysis timestamp.

4. ai_knowledge: Vector knowledge base containing curated diagnostic resolutions.

- id (INTEGER, Primary Key): Unique knowledge entry ID.

- event_id (VARCHAR): Associated Windows event identifier.

- description (VARCHAR): Technical failure description.

- embedding (VECTOR(384)): Dense vector embedding (384 dimensions) generated via FastEmbed BAAI/bge-small-en-v1.5 (with SentenceTransformer all-MiniLM-L6-v2 fallback).

- solution_json (JSON): Recommended administrative mitigation procedures.

5. hardware_inventories: Catalogs deep physical hardware configurations linked to an asset.

- inventory_id (UUIDv4, Primary Key): Inventory snapshot identifier.

- asset_id (UUIDv4, Foreign Key): Linked asset entity.

- cpu_sku_model (VARCHAR): Microprocessor vendor and model name.

- total_ram_mb (INTEGER): Total physical memory capacity.

- storage_topology (JSONB): Attached physical disks, controller types, and partition volumes.

6. audit_logs: Tamper-evident operational audit journal.

- log_id (UUIDv4, Primary Key): Transaction identifier.

- actor_id (UUIDv4, Nullable): Operating user or service account.

- asset_id (UUIDv4, Nullable): Target asset affected.

- action_verb (VARCHAR): Operational action executed.

- performed_at (TIMESTAMPTZ): UTC execution timestamp.

- immutable_payload (JSONB): State snapshot before and after mutation.

Entity Relationship Diagram

Source listing mermaid

```
erDiagram
infrastructure_assets ||--o{ windows_event_logs : "emits (1:N)"
infrastructure_assets ||--o{ hardware_inventories : "possesses (1:N)"
infrastructure_assets ||--o{ audit_logs : "records_mutation (1:N)"
infrastructure_assets ||--o{ analysis_history : "referenced_in_metadata (1:N)"
windows_event_logs ||--o{ analysis_history : "linked_via_event_source_id (1:1..N)"
ai_knowledge ||--o{ analysis_history : "semantic_match (1:N)"
```

```
infrastructure_assets {
uuid asset_id PK
string hostname
inet canonical_ip
string cryptographic_fingerprint UK
string lifecycle_state
smallint current_compliance_score
timestamptz created_at
timestamptz updated_at
}
windows_event_logs {
uuid log_id PK
uuid asset_id FK
timestamptz occurrence_time
int event_id
string severity_level
jsonb evtx_metadata
}
analysis_history {
int id PK
string event_id
string provider
string parse_method
string description
string ai_summary
json solution_summary
json event_metadata
json search_results
datetime created_at
}
ai_knowledge {
int id PK
string event_id
string description
vector_384 embedding
json solution_json
}
hardware_inventories {
uuid inventory_id PK
uuid asset_id FK
string cpu_sku_model
int total_ram_mb
jsonb storage_topology
}
audit_logs {
uuid log_id PK
uuid actor_id FK
uuid asset_id FK
string action_verb
timestamptz performed_at
jsonb immutable_payload
}
```

3.4 Portable USB Auditor Design

The Portable USB Auditor (clients/usb_auditor/) is a standalone auditing utility designed to execute on isolated Windows endpoints.

Core Architectural Decisions

- Self-Contained CPython Runtime: The package incorporates an embedded, portable CPython 3.14.3 amd64 interpreter (downloaded as python-3.14.3-embed-amd64.zip during packaging). The target host requires no preinstalled Python, pip, Git, Docker, or external runtimes.

- Batch Launcher (Run-EIMS-Audit.bat): Field technicians execute audits simply by double-clicking the batch script. The launcher establishes relative working paths, executes runtime integrity preflights, and invokes the Python scanning engine.

- Local Evidence Artifacts: In portable execution mode, the configuration variable EIMS_AUTO_SYNC defaults to false. The scanning engine structures all collected telemetry into a structured JSON report written directly to the removable drive at reports/audit_<hostname>_<timestamp>.json.

- Package Integrity Verification: The package includes a cryptographic manifest (manifest.sha256) containing SHA-256 hashes of all internal audit scripts and dependencies. The launcher verifies manifest integrity prior to execution to detect media corruption or file tampering.

- Strict Read-Only Posture: Runs from a USB drive with a bundled runtime; uses read-only queries and writes its output to configured report/log directories. No persistent agent installation is required.

- BitLocker Secret Exclusion Boundary: The scanner interrogates BitLocker drive encryption via CIM, recording posture attributes (protection_status, volume_status, encryption_percentage, encryption_method, recovery_protector_present, and recovery_protector_count). The collection script strictly excludes recovery keys; parameters such as RecoveryPassword, recovery_key, and plaintext protector secrets are never extracted or stored. A case-insensitive secret stripping function cleans all memory structures prior to report generation.

Builder Safety Controls

The USB Auditor packaging tool (tools/build_usb_package.py) incorporates stringent drive safety checks:

- The builder requires explicit, manual selection of a verified removable drive letter.

- It executes no automatic formatting and no disk repartitioning commands.

- It validates filesystem permissions and verifies available drive capacity prior to copying the runtime bundle.

3.5 Endpoint Evidence Collection

The USB Auditor scanning engine gathers evidence across seven operational categories:

1. System & OS Information: Hostname, domain/workgroup, Windows edition, kernel build, install date, and system uptime.

2. Hardware Configuration: CPU architecture, core count, physical RAM modules, motherboard serial number, BIOS version, and physical storage drives.

3. Network Configurations: Active network adapters, MAC addresses, IPv4/IPv6 addresses, subnet masks, default gateways, and DNS servers.

4. Security & Hardening Posture: Windows Defender antimalware status and definition age, Windows Firewall profile enforcement (Domain, Private, Public), Windows Update service status, and BitLocker encryption posture.

5. Running Services: Windows services inventory, highlighting critical infrastructure daemons.

6. Compliance Scoring: Local evaluation of baseline hardening metrics yielding a local preliminary compliance score (0–100).

7. Windows Event Log Evidence: Scoped extraction of operational event logs.

Event Collection Constraints

To balance diagnostic visibility with storage and performance constraints:

- Channels: The portable graduation workflow intentionally scopes Windows Event Log collection to the System and Application channels, focusing diagnostic evidence acquisition directly on core operating system reliability, hardware health, and enterprise software execution.

- Temporal Window: Defaults to the preceding 24 hours of operational history.

- Record Threshold: Default maximum of 500 events across the combined query (-MaxEvents 500 across System and Application, filtering Critical, Error, and Warning events), preventing memory exhaustion on unstable hosts experiencing severe log flooding.

3.6 Offline Report Schema and Ingestion

Ingestion of offline evidence into the centralized platform is handled via the canonical API endpoint:

Source listing http

```
POST /api/v1/assets/import-report
```

The endpoint accepts a multipart form upload containing the raw JSON report emitted by the USB Auditor.

End-to-End Ingestion Flow

1. Payload Reception & Deserialization: The endpoint receives the multipart file stream, deserializes the JSON content, and validates schema conformance using Pydantic models.

2. Asset Entity Upsert: The repository extracts the asset's cryptographic fingerprint (or networking hostname) and performs a PostgreSQL relational upsert:

- If the asset exists, its hardware inventory, network configuration, and compliance score are updated.

- If the asset is novel, a new infrastructure_assets entity is created with state Compliant (if score >= 70) or NonCompliant.

3. Event Normalization: The pipeline extracts the event_logs array. Individual events are validated for structural integrity, mapping fields to canonical schema parameters (event_id, severity_level, occurrence_time, channel, provider, record_id, message).

4. Sequential Deduplication: The service computes SHA-256 deduplication keys for each normalized event and queries existing records in PostgreSQL to eliminate duplicate entries.

5. Batch Persistence: Novel, non-duplicate events are inserted into windows_event_logs within an atomic database transaction.

6. Severity Prioritization: Novel events are routed into a priority ranking queue, sorting records by severity (Critical > Error > Warning > Information).

7. Bounded AI Analysis: The top candidates (strictly bounded to a maximum of 10 events) are dispatched to the AI Analyzer service.

8. Analysis History Persistence: Generated diagnostic summaries and mitigation recommendations are committed to analysis_history with complete asset and event provenance.

Offline Evidence Ingestion Sequence

Source listing mermaid

```
sequenceDiagram
autonumber
actor Tech as Field Technician / Operator
participant Target as Windows Target Host
participant USB as Portable USB Auditor
participant API as FastAPI Ingestion Gateway
participant Repo as Asset Repository
participant Pipeline as Import Pipeline (Dedup & Priority)
participant PG as PostgreSQL Datastore
participant AI as AI Log Analyzer
participant UI as Next.js Dashboard
Tech->>Target: Insert USB & Execute Run-EIMS-Audit.bat
Target->>USB: Interrogate System, CIM & Event Logs (System/App)
USB->>USB: Generate Local JSON Report (reports/audit_*.json)
Tech->>API: POST /api/v1/assets/import-report (Multipart JSON)
API->>Repo: upsert_offline_report(report_data)
Repo->>PG: Upsert infrastructure_assets & hardware_inventories
PG-->>Repo: Return Canonical asset_id
API->>Pipeline: ingest_report_events(db, asset_id, report_data)
Pipeline->>Pipeline: Normalize Events & Compute SHA-256 Dedup Keys
Pipeline->>PG: Query Existing _reporter_dedup Keys for asset_id
PG-->>Pipeline: Return Existing Keys Set
Pipeline->>Pipeline: Filter Duplicates & Sort by Severity (Critical > Error)
Pipeline->>PG: Batch Insert Novel WindowsEventLog Rows
Pipeline->>AI: Dispatch Top Candidates (Max 10 Events)
AI->>PG: Vector Query ai_knowledge (Cosine Distance)
PG-->>AI: Return Solution Snippets
AI->>AI: Synthesize Diagnostic Summary & Remediation
AI->>PG: Insert AnalysisHistory (with asset_id & event_source_id)
API-->>Tech: HTTP 200 OK (AssetResponse + Ingestion Summary)
Tech->>UI: Navigate to Endpoint Investigation View
UI->>API: GET /api/v1/telemetry/winlogs?asset_id=...
UI->>API: GET /api/v1/history?asset_id=...
UI-->>Tech: Render Ingested Events & Inline AI Diagnostic Findings
```

3.7 Sequential Deduplication Design

To prevent duplicate operational records during recurring offline audits, EIMS implements an application-level cryptographic deduplication mechanism.

Deduplication Key Derivation

For each incoming event, the pipeline computes a deterministic SHA-256 hash across five immutable operational attributes:

DedupKey = SHA-256(asset_id | channel | provider | record_id | occurrence_time)

This composite key is stored within the event's evtx_metadata JSONB column under the key _reporter_dedup.

Query Evaluation and Filtering

During ingestion, the service extracts all deduplication keys from the incoming payload and queries PostgreSQL:

Source listing sql

```
SELECT evtx_metadata->>'_reporter_dedup'
FROM windows_event_logs
WHERE asset_id = :asset_id
AND evtx_metadata->>'_reporter_dedup' IN (:keys);
```

Matching keys returned by the query are classified as duplicate events and skipped. Only novel keys proceed to database insertion.

Architectural Limitation and Concurrency Boundary

It is vital to state the architectural boundary of this implementation: deduplication is enforced at the application level during sequential processing. It is not enforced by a PostgreSQL table-level UNIQUE constraint across the JSONB path. Consequently, while this mechanism reliably prevents duplicate record insertion during normal sequential report imports, concurrent parallel uploads of identical reports across multiple threads could experience race conditions. Deduplication is enforced at the application level during sequential processing. Database-level unique constraints and concurrency control across concurrent imports remain future work.

3.8 Event Prioritization

To prevent analytical resource exhaustion and focus operator attention on critical system failures, the ingestion pipeline implements deterministic severity prioritization:

1. Priority Stratification: Events are assigned numerical priority ranks based on normalized severity strings:

- Critical $\rightarrow$ Priority 3

- Error $\rightarrow$ Priority 2

- Warning $\rightarrow$ Priority 1

- Information $\rightarrow$ Priority 0

2. Stable Sorting: Candidate events are sorted in descending order of priority. To ensure reproducible behavior, identical priority tiers preserve their original incoming sequence.

3. Analytical Workload Cap: The implementation enforces a strict upper bound of at most 10 newly ingested events analyzed per import run.

4. Duplicate Analysis Prevention: The pipeline queries analysis_history to verify whether candidate deduplication keys have previously undergone analysis, skipping already-analyzed events.

3.9 AI-assisted Analyzer Integration

The EIMS AI Analyzer (backend/domain/analyzer/) provides contextual diagnostic assistance for prioritized operational failures.

Semantic Retrieval Pipeline

1. Event Parsing: The analyzer constructs a normalized event description combining Event ID, Provider name, Channel, and extracted message content.

2. Local Vector Embedding: The system generates a 384-dimensional vector embedding using FastEmbed (primary model BAAI/bge-small-en-v1.5, with SentenceTransformer all-MiniLM-L6-v2 fallback). Analysis evaluates curated answers first, then retrieval from ai_knowledge, optional Gemini synthesis when configured, retrieved answers, and web-derived fallback.

3. pgvector Similarity Query: The embedding is queried against the ai_knowledge table using PostgreSQL's cosine distance operator (<=>):

Source listing sql

```
SELECT id, event_id, description, solution_json, (embedding <=> :query_vector) AS distance
FROM ai_knowledge
ORDER BY distance ASC
LIMIT 3;
```

4. Synthesis of Mitigation Advice: The retrieved solution snippets, combined with curated catalog heuristics, are processed by the diagnostic synthesis service, producing structured mitigation advice, relevant Microsoft documentation links, and diagnostic confidence scores.

5. Non-Fatal Error Isolation: The entire analytical pipeline is wrapped in non-fatal exception handling. If embedding generation or synthesis encounters an error (such as a timeout or database contention), the failure is logged, but the parent transaction safely commits the persisted windows_event_logs records. Committed event rows are retained when a later analysis attempt fails.

3.10 Analysis Provenance

Every finding generated by the AI Analyzer records explicit audit provenance within the analysis_history.event_metadata JSON column:

Source listing json

```
{
"source_type": "USB_OFFLINE_COLLECTION",
"source_subtype": "WINDOWS_EVENT_LOG",
"asset_id": "00000000-0000-0000-0000-000000000000",
"event_source_id": "11111111-1111-1111-1111-111111111111",
"channel": "System",
"provider": "Service Control Manager",
"record_id": 14205,
"occurrence_time": "2026-09-28T08:14:22.000Z",
"_reporter_dedup": "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855"
}
```

- source_type: Identifies the collection methodology (USB_OFFLINE_COLLECTION).

- asset_id: Formally links the analysis record to the parent infrastructure asset.

- event_source_id: Establishes a JSON-embedded foreign reference pointing to the specific windows_event_logs.log_id record that triggered the analysis.

- parse_method: Recorded as USB_OFFLINE_COLLECTION:USB_OFFLINE_COLLECTION in the table's classification column.

Because analysis_history was designed to accommodate polymorphic analysis types across both operational event streams and ad-hoc queries, event_source_id is maintained as a structured attribute within event_metadata rather than a rigid relational SQL foreign key constraint. This design preserves complete audit traceability back to originating event log entries while maintaining schema flexibility.

3.11 Backend API Design

The FastAPI backend exposes standardized, RESTful HTTP interfaces conforming to OpenAPI 3.1 specifications.

Core Interface Endpoints

- POST /api/v1/assets/import-report: Canonical ingestion endpoint accepting multipart JSON reports. Returns an AssetResponse containing the updated asset entity and an event_ingestion summary object.

- GET /api/v1/assets: Retrieves paginated lists of registered infrastructure assets, supporting filters by hostname, lifecycle state, and compliance score threshold.

- GET /api/v1/assets/{asset_id}: Returns deep asset details, including linked hardware inventories and active security configurations.

- GET /api/v1/telemetry/winlogs: Queries ingested Windows Event Log records, accepting query parameters for asset_id, event_id, severity, and ISO-8601 temporal bounds (from, to).

- GET /api/v1/history: Retrieves historical AI analysis records, supporting asset-scoped filtering via the asset_id query parameter (GET /api/v1/history?asset_id=...).

- GET /api/v1/search: Executes multi-domain search across assets, event logs, and audit records, supporting global keyboard navigation.

- GET /api/v1/timeline: Returns a unified chronological timeline aggregating audit logs, telemetry metrics, and Windows event logs.

- GET /api/v1/health: Evaluates backend health and verifies active connectivity to PostgreSQL, Redis, and MinIO datastores.

Error Handling Architecture

The backend architecture establishes an RFC 7807 Problem Details exception framework (application/problem+json) for schema validation failures (RequestValidationError) and domain exceptions (EIMSProblemException), while standard endpoint HTTP status exceptions return structured detail payloads. When validation or domain exceptions occur, the gateway renders structured JSON envelopes:

Source listing json

```
{
"type": "https://eims.internal/errors/invalid-report",
"title": "Invalid Offline Report Format",
"status": 400,
"detail": "Failed to parse offline report: missing required cryptographic fingerprint.",
"instance": "/api/v1/assets/import-report",
"tracking_uuid": "c3b8a1f4-9b2e-4d5a-8f1e-2a3b4c5d6e7f"
}
```

3.12 Investigation Dashboard

The frontend interface (clients/dashboard/) is developed with Next.js, React, and TypeScript, styled using Tailwind CSS design tokens.

Interface Views

1. Endpoints Management View (/endpoints): Presents a comprehensive data table of all registered infrastructure assets, displaying real-time compliance badges, operating system versions, and quick-filter toggles.

2. Asset Inspection Detail View (/endpoints/[id]): Deep diagnostic portal displaying:

- System specification cards (CPU SKU, memory capacity, storage layout).

- Network interface bindings and canonical IP assignments.

- Security and compliance posture meters (BitLocker encryption status, Windows Defender recency, Firewall enforcement).

- Event Evidence Table: Interactive event viewer rendering ingested System and Application logs with severity badges, channel filters, and record timestamps.

- Inline AI Findings: Dynamic panel rendering synthesized diagnostic findings, root-cause explanations, and mitigation action checklists corresponding to the specific asset.

3. Global Search Palette (Ctrl+K): Keyboard-driven search interface querying assets, hostnames, IP addresses, and event identifiers across nine registered search providers .

4. Unified Operational Timeline (/timeline): Synchronized chronological stream correlating configuration changes, telemetry spikes, and critical event failures.

Note: Formal browser runtime verification and UI rendering validation results are presented in Chapter 4.

3.13 Authentication and Security Design

EIMS enforces strict security and authentication boundaries across all interfaces.

Dual Authentication Modes

The backend platform supports two operational authentication modes configured via the EIMS_AUTH_MODE environment variable:

- demo Mode: Used for local development, rapid evaluation, and automated end-to-end testing in isolated environments. The authentication dependency (get_current_user) bypasses token validation and returns a standardized demo identity with administrative access.

- secure Mode: Enforces mandatory cryptographic token verification. Incoming requests must carry a valid HTTP Authorization: Bearer <JWT> header signed with a shared secret using the HS256 algorithm. Requests lacking valid tokens are rejected with HTTP 401 Unauthorized errors.

Removal of Privileged Static Tokens

In early development iterations, client-side administrative pages utilized a hardcoded administrative token (EIMS-ADMIN-TOKEN). During security hardening, this vulnerability was eliminated:

- All privileged static tokens were completely removed from frontend TypeScript source files.

- Administrative write operations (such as manual asset state changes) are protected on the backend using the verify_admin_token / require_admin_for_write dependency, ensuring that privileged actions require authenticated server-side sessions.

Trusted Local Sandbox Limitation

In the current cooperative education release, demo mode operates under the explicit architectural assumption of a trusted local execution boundary. Deploying the system in untrusted or multi-tenant production networks requires activating secure mode and provisioning dedicated API gateway credentials.

3.14 Observability and Supporting Infrastructure

The containerized infrastructure architecture is orchestrated via Docker Compose, incorporating:

- PostgreSQL 16: Configured with relational write-ahead logging (WAL) and the pgvector extension.

- PgBouncer: Configured in transaction pooling mode to manage backend database connections efficiently.

- Redis 7: Running Alpine Linux containers configured with volatile-lru eviction policies.

- MinIO: S3-compliant object store hosting unstructured hardware documentation imagery.

- Prometheus & Grafana: Continuous metrics scraping infrastructure polling backend /metrics endpoints.

3.15 Development and Validation Methodology

Platform reliability and architectural compliance were validated through a structured, multi-tier testing methodology:

1. Automated Unit Testing: Unit test suites (executed via pytest) validating pure helper functions, deduplication key derivation, event normalization, priority queue sorting, and case-insensitive BitLocker secret stripping.

2. Integration Testing: API integration tests executing against isolated FastAPI test clients, asserting HTTP status codes, schema validation, and error envelopes.

3. Live PostgreSQL Persistence Validation: Database integration suites executing against live PostgreSQL container instances, validating JSONB containment operators (->>, @>) and pgvector cosine similarity syntax.

4. Frontend Static Typing & Production Build: Comprehensive TypeScript compilation checks (npx tsc --noEmit) and Next.js production bundle compilation (npm run build), asserting zero type errors and zero broken imports.

5. USB Packager Preflight Validation: Verification of the USB Auditor packaging script, asserting removable drive preflight safety, file copy completeness, and SHA-256 manifest generation.

6. Field Endpoint Validation: Execution of the portable auditor on physical and virtual Windows test workstations, asserting zero runtime crashes and valid report generation.

7. Production-Representative Windows Server Validation: Execution of the auditor on an authorized enterprise Windows Server host, capturing real-world operational event logs.

8. Sequential Duplicate Ingestion Validation: Verification of the ingestion pipeline by executing sequential re-imports of identical field reports, asserting that duplicate events are filtered without error.

9. Live API and Browser UI Verification: Verification of live API endpoints (GET /api/v1/telemetry/winlogs and GET /api/v1/history?asset_id=...) and interactive verification of the web dashboard rendering event evidence and AI findings without error.

Note: Measured test execution values, pass rates, and empirical field results are presented in Chapter 4.

3.16 Data Protection and Evidence Handling

To ensure compliance with corporate confidentiality agreements and academic integrity standards, EIMS enforces strict data protection rules:

- Public Repository Confidentiality Policy: No proprietary enterprise data, customer identifiers, real IP addresses, MAC addresses, hardware serial numbers, user account names, internal domain names, or raw system logs may be committed to public version control repositories.

- Strict Anonymization of Academic Artifacts: In all academic documentation, reports, and presentation materials, real enterprise infrastructure entities are replaced with standardized aliases (e.g., ASSET-01, Endpoint A, Windows Server A).

- BitLocker Secret Exclusion: The collection utilities and ingestion parsers strictly prohibit the acquisition, transmission, or persistence of BitLocker Recovery Keys or plaintext passwords.

- Physical Separation of Evidence: Raw, unredacted field audit reports and full-resolution unredacted screenshots are archived in secure, local private storage directories outside the Git repository. Only sanitized, anonymized artifacts are referenced within project documentation.

3.17 Design Limitations

To maintain academic and professional honesty, the architectural boundaries and known limitations of EIMS are explicitly documented:

- Windows-First Scope: The current implementation of the portable collector and event ingestion pipeline is specialized for the Microsoft Windows operating system ecosystem. Cross-platform support for Linux distributions or macOS endpoints is deferred to future work.

- Manual Physical Transport (Sneaker-Net): In air-gapped environments, transferring the offline JSON evidence report from the target machine to the centralized EIMS platform requires manual physical transport of the USB drive by an operator.

- Application-Level Sequential Deduplication: Event deduplication is enforced at the application level during sequential processing. Simultaneous parallel imports of identical reports across concurrent threads could experience race conditions due to the absence of database-level unique constraints on JSONB fields.

- Bounded AI Workload: The AI Analyzer processes at most 10 prioritized events per batch import. While this safeguards system stability and controls computational latency, events outside the selected subset are not automatically analyzed by re-importing the same report; a separate review or reanalysis workflow would be future work.

- Non-Destructive Read-Only Operation: EIMS deliberately omits automated host remediation capabilities. The system identifies failures and suggests mitigation procedures, but does not execute automated configuration changes on monitored endpoints.

- Operational Triage vs. Certified Forensics: EIMS is designed as an infrastructure management and operational triage aid. It is not certified as a court-admissible digital forensics acquisition platform.

- AI as Assistive Triage: AI-generated diagnostic explanations and remediation plans represent probabilistic recommendations derived from semantic vector matching; they do not constitute infallible or legally binding root-cause guarantees.

- Prototype Deployment Architecture: The current graduation release is deployed via single-instance containers orchestrated with Docker Compose; it does not implement enterprise high-availability clustering, distributed multi-region failover, or enterprise Single Sign-On (SSO) federation.

3.18 Chapter Summary

This chapter detailed the comprehensive system design and implementation of the Enterprise Infrastructure Management System (EIMS). The discussion formalized functional and non-functional requirements, mapped operational personas, distinguished the canonical PRD baseline from the implemented graduation scope, articulated the hybrid modular monolith architecture with complete component and sequence diagrams, and detailed the persistent relational and vector data models. Furthermore, the self-contained portable USB Auditor, the offline ingestion pipeline, application-level sequential deduplication, severity prioritization, and RAG-based AI analysis were thoroughly analyzed. Finally, the Next.js web dashboard, authentication boundaries, multi-tier validation methodology, and data protection policies were documented, concluding with an honest appraisal of system design limitations. With the system design and implementation

established, Chapter 4 presents the empirical validation results, test execution metrics, and real-world field verification evidence.

Source docs/report/CHAPTER_4_RESULTS_AND_VALIDATION.md

Chapter 4

Results and Validation

Current graduation revision and evidence boundary

Current source audit: 1 November 2026 — 2a0d9031297ac1063a0f327a8a76c455bb5da315 (PR #10). Next.js 16.3.0 / React 19.2.8. Engineering CI: 391 Python tests (299 Analyzer hermetic + 46 synthetic PostgreSQL + 46 USB isolated), run 41230498604, not project-wide certification. DEV 12 DRAFT, HELDOUT 5 reserved, claims 61, genuine human reviews 0. Diagnostic accuracy NOT_MEASURED; secure HTTPS browser BLOCKED_NOT_TESTED; current USB/production acceptance NOT_TESTED. No main merge.

Current October engineering and scientific results

Evidence layer	Current result	Limit
Analyzer hermetic + synthetic PostgreSQL CI	299 + 46 + 46 = 391 PASS at source above, CI run 41230498604	Does not cover all modules or a deployed system
Counterexamples	8/8 PASS	Synthetic regression, not accuracy
Frozen M0/M1/Full	12 DEV executions per mode; Full emits 61 claims	All DRAFT; 0 human-reviewed cases/claims
Scientific metrics	Accuracy, semantic support precision and unsupported-claim rate NOT_MEASURED	Zero reviewed denominator; no generalization claim
Secure HTTPS/USB/production	BLOCKED_NOT_TESTED / NOT_TESTED / NOT_CERTIFIED	Historical A9 is a different boundary

Current snapshot: evaluation/analyzer/results/20261101-priority-full_investigation; source/input/annotation/prediction hashes and review bindings preserved. New bounded project-wide smoke results are in docs/research/PROJECT_WIDE_READINESS_20261031.md; do not add their counts to the 391-test CI claim. Historical validation below was assembled on 29 September 2026 at editorial baseline c38bef97a5f18e85d4338108cb18c48a57d7e1a4; individual field-run SHAs/dates are not newly verified. 49 stored events, 10 findings and A9 PASS remain historical observations, not October Secure Mode acceptance.

Historical prototype material (retained for provenance)

This chapter presents the empirical validation results, experimental observations, test suite execution metrics, and field verification evidence for the Enterprise Infrastructure Management System (EIMS). The primary objective is to substantiate the functional correctness, data provenance, deduplication resilience, and user interface delivery of the platform based strictly on recorded engineering evidence.

4.1 Validation Objectives

The verification of EIMS was structured to demonstrate that the implemented architecture satisfies the core engineering objectives defined in Chapter 1. The validation methodology focused on eight primary dimensions:

1. Automated Regression and Build Integrity: Verifying that core backend services, ingestion routines, and frontend client code compile without type errors and pass automated regression suites.

2. Portable Evidence Collection Integrity: Validating the self-contained execution, non-destructive behavior, and JSON report generation of the Portable USB Auditor on target Windows hosts.

3. Asset Registry and Ingestion Reliability: Demonstrating the relational ingestion of offline reports and the persistent linkage of asset identity and configuration state.

4. Operating System Event Persistence: Validating the structured persistence of Windows Event Logs across the System and Application channels with complete collection metadata.

5. Sequential Deduplication Behavior: Demonstrating that identical, sequential re-imports of event evidence do not generate redundant database records.

6. Severity Prioritization and Bounded AI Triage: Verifying that incoming operational events are ranked deterministically by severity and that analytical workloads are strictly bounded to the configured threshold.

7. Traceable Audit Provenance: Validating end-to-end provenance linking persisted AI diagnostic findings back to their originating Windows Event Log records and parent asset entities.

8. Operational Web Interface Delivery (A9 Gate): Confirming via browser runtime verification that asset-linked event evidence and active AI diagnostic findings are successfully presented on the web dashboard.

Validation activities combined hermetic automated testing with field verification conducted on authorized enterprise Windows endpoints.

4.2 Validation Environment

Field validation was executed within an authorized enterprise test environment using production-representative server infrastructure. To protect proprietary enterprise data in accordance with project confidentiality policies, all infrastructure entities are identified in this report using academic aliases.

Component / Layer	Validation Specification & Environment Configuration
Backend API Gateway	FastAPI running on Python 3.12 within Docker container
Frontend Web Dashboard	Next.js 14 / React 18 / TypeScript running within Docker container
Relational Datastore	PostgreSQL 16 with pgvector extension enabled
In-Memory Cache & Broker	Redis 7 (Alpine Linux)
Binary Object Storage	MinIO S3-compatible local object store
Connection Pooling	PgBouncer in transaction pooling mode
Portable Collector Runtime	CPython 3.14.3 amd64 embedded runtime (python-3.14.3-embed-amd64.zip)
Field Validation Target	ASSET-01 (Authorized enterprise Windows Server host)
Monitored Event Channels	Windows System and Application channels
Event Collection Window	Preceding 24 hours of operational history
Event Acquisition Bound	Default maximum 500 events across combined System/Application query

Note: While field validation was conducted on the specified Windows Server host, EIMS architecture supports diverse Windows compute endpoints, including Windows 11 Enterprise and Windows 10 Pro workstations.

4.3 Automated Regression and Build Validation

Prior to field deployment and downstream data verification, automated test suites and compiler checks were executed across the codebase. Validation was bounded to the active feature set and integration gates.

Validation Suite / Gate	Tooling / Command Executed	Observed Result	Engineering Significance
Event Evidence Ingestion Suite	pytest tests/test_sprint13_evidence_ingestion.py	37 passed, 0 failed, 4 warnings	Reported hermetic ingestion tests (mocked database/analyzer); separate historical/live PostgreSQL checks.
USB Auditor Sync Suite	pytest clients/usb_auditor/tests/test_auto_sync.py	18 passed, 0 failed	Validates bounded HTTP upload, fallback to local storage, and configuration flag parsing.
Frontend Static Typing	npx tsc --noEmit (Dashboard root)	PASS (Exit Code 0)	Asserts zero TypeScript compilation errors, verifying strict interface contract alignment with backend models.
Frontend Production Build	npm run build (Next.js App Router)	PASS (Exit Code 0)	Confirms static page pre-rendering, bundle tree-shaking, and zero broken module imports.
Git Whitespace & Syntax Gate	git diff --check	PASS (Exit Code 0)	Asserts zero trailing whitespace, merge conflict remnants, or line ending corruption across all committed files.

In the executed regression scope, no functional failure was observed. Automated test coverage confirmed that core helper algorithms—specifically cryptographic key derivation and JSON normalization—behave deterministically across edge-case permutations.

4.4 Portable Evidence Collection Validation

The operational behavior of the Portable USB Auditor (clients/usb_auditor/) was previously validated during earlier on-site field testing sessions. The latest validation session focused primarily on downstream database persistence and browser interface delivery, relying on the evidence artifacts acquired during those prior field sessions.

During earlier field acquisition testing, the portable utility demonstrated the following operational capabilities:

- Zero-Dependency Execution: The auditor executed successfully on target hosts running Windows Server without requiring preinstalled Python interpreters, package managers (pip), Git, Docker, or external

application dependencies. The CPython 3.14.3 amd64 embedded runtime executed cleanly from the removable USB medium.

- Batch Script Initiation: The batch launcher (Run-EIMS-Audit.bat) established relative execution paths, completed runtime preflight checks, and executed the collection sequence upon double-click invocation.

- Local Evidence Generation: The scanning engine generated structured JSON reports stored under reports/audit_<hostname>_<timestamp>.json on the USB drive. Local evidence creation was self-contained, preserving data on disk regardless of subsequent network upload success.

- Package Manifest Integrity: The SHA-256 manifest verification routine successfully validated internal script hashes prior to scanning, ensuring that corrupted or tampered files halted execution.

- Non-Destructive Execution: System interrogation was performed exclusively via read-only CIM cmdlets and event querying utilities, using read-only queries and writing output to configured report and log directories without persistent host software installation.

[FIGURE PLACEHOLDER — Portable USB Auditor Batch Launcher Console Output]

Suggested caption: Figure 4.1. Console output generated during the execution of Run-EIMS-Audit.bat on a target Windows host.

Note: Identifiers and host paths have been anonymized.

4.5 Asset Registry and Compliance Results

Ingestion of offline report data into the central platform was verified by inspecting the live relational database and asset management interfaces for the selected enterprise host (ASSET-01).

Verification confirmed:

1. Canonical Asset Entity: ASSET-01 was indexed within the infrastructure_assets table with an assigned UUIDv4 canonical identifier.

2. Metadata Population: Hostname, primary network interface IP address, operating system build, and total physical memory were successfully parsed from the report and persisted.

3. Security Posture Evaluation: Security configuration flags—including Windows Firewall profile status, Windows Defender antimalware definition timestamps, and BitLocker volume encryption status—were successfully mapped to the asset entity.

4. Secret Exclusion: In accordance with the security design established in Chapter 3, zero BitLocker Recovery Passwords or plaintext protector secrets were present in the database records or API responses.

[FIGURE PLACEHOLDER — Asset Overview and Security Posture View]

Suggested caption: Figure 4.2. Operational Dashboard detail view displaying hardware specifications and security compliance posture for ASSET-01.

Note: Identifiers have been anonymized.

4.6 Windows Event Evidence Results

Downstream verification evaluated the persistence and retrieval of operating system event evidence captured from ASSET-01.

The live API endpoint was queried using an asset-filtered request:

GET /api/v1/telemetry/winlogs?asset_id=<ASSET-01>

Observed Field Metrics

- HTTP Response Status: 200 OK

- Total Persisted Records Returned: 49 rows

- Asset Linkage: 100% of returned records referenced the canonical asset_id of ASSET-01.

- Channel Distribution: The 49 ingested events comprised:

- System Channel: 34 records

- Application Channel: 15 records

- Temporal Window: The events spanned the preceding 24-hour collection window configured on the auditor.

- Metadata Fields Confirmed: Every returned record contained valid values for channel, provider, record_id, occurrence_time, and severity_level.

- Deduplication Envelope: The _reporter_dedup SHA-256 composite hash was populated across all 49 rows inside the evtx_metadata JSONB column.

It is vital to distinguish between raw collected evidence and events selected for artificial intelligence analysis. The 49 persisted rows represent the raw operational events collected from the endpoint; only a prioritized subset was subsequently routed to the AI Analyzer.

[FIGURE PLACEHOLDER — Windows Event Evidence Table View]

Suggested caption: Figure 4.3. Next.js Event Evidence table rendering 49 persisted Windows Event Log records filtered by ASSET-01 across System and Application channels.

Note: Identifiers have been anonymized.

4.7 Sequential Deduplication Validation

Deduplication behavior was evaluated based on the application-level sequential fingerprinting algorithm specified in Section 3.7. The deduplication key for each event is derived deterministically:

DedupKey = SHA-256(asset_id | channel | provider | record_id | occurrence_time)

Previously Validated Ingestion Behavior

In earlier test suite and field ingestion experiments where identical JSON reports were submitted sequentially to /api/v1/assets/import-report:

1. Initial Import: The pipeline normalized incoming events, queried existing keys (returning an empty set for novel assets), and inserted all valid events into windows_event_logs.

2. Sequential Re-Import: When the exact same report was uploaded a second time:

- The pipeline computed identical SHA-256 keys for all events.

- The query against evtx_metadata["_reporter_dedup"].astext matched 100% of the incoming keys against previously committed records.

- The ingestion counter reported: events_received = N, events_new = 0, events_duplicate = N.

- Zero redundant records were written to windows_event_logs.

- Downstream AI analysis was skipped for duplicate events, preventing redundant embedding computation and database churn.

Operational Boundaries

It is an essential finding of this project that deduplication is enforced at the application layer during sequential processing. Because PostgreSQL does not enforce a unique database constraint across the JSONB deduplication path, concurrent parallel uploads of identical reports could experience race conditions. Furthermore, the latest on-site validation session focused exclusively on verifying existing downstream database rows and did not re-execute a live duplicate report upload.

4.8 Event Prioritization and Analyzer Results

The prioritization and triage pipeline was evaluated against the persisted analytical records associated with ASSET-01.

Prioritization Rule Execution

The ingestion engine ranks events according to severity:

Critical (Priority 3) > Error (Priority 2) > Warning (Priority 1) > Information (Priority 0)

In accordance with the bounded workload constraint established in Section 3.8, the implementation limits analysis to at most 10 newly ingested events per import run.

Field Analysis Findings

Inspection of the persistent database for ASSET-01 revealed:

- Total Analysis Records Linked to Asset: Exactly 10 analysis_history records.

- Workload Bounding Alignment: The existence of exactly 10 analysis records corresponds directly to the configured cap of 10 events per import, confirming that the analytical service processed the maximum allowable batch size without unboundedly analyzing the entire set of 49 collected events.

- Diagnostic Content: Each analysis record contained a synthesized diagnostic summary, contextual remediation suggestions, and confidence metrics derived from cosine distance matching against the vector knowledge base.

AI-generated summaries functioned as operational investigation aids, providing systems engineers with rapid context and relevant administrative references without asserting autonomous root-cause certainty.

4.9 AnalysisHistory and Provenance Results

Traceability from analytical findings back to originating event evidence was validated by querying the filtered history API:

GET /api/v1/history?asset_id=<ASSET-01>

Observed Metrics and Query Integrity

- HTTP Status: 200 OK (Clean query execution with zero server errors).

- Filtered Records for Asset: 10 records.

- Unfiltered Total Records in Database: 21 records across all historical test imports.

- Query Precision: The API correctly filtered records by the asset identifier embedded in the metadata, returning only records belonging to ASSET-01.

Provenance Attribute Verification

Every analysis record returned for ASSET-01 was inspected for provenance compliance:

1. source_type: Confirmed as "USB_OFFLINE_COLLECTION" across all 10 records.

2. parse_method: Confirmed as "USB_OFFLINE_COLLECTION:USB_OFFLINE_COLLECTION".

3. asset_id: Correctly populated, referencing the canonical asset entity.

4. event_source_id: Populated within the event_metadata JSON column, linking directly to the primary key (log_id) of the originating windows_event_logs record.

5. _reporter_dedup: Populated within event_metadata, enabling bidirectional traceability between raw event hashes and analytical conclusions.

This verification confirms that event_source_id operates as an effective JSON-embedded audit link, ensuring that any finding displayed to an operator can be traced backward to the originating raw event log.

[FIGURE PLACEHOLDER — Filtered History JSON API Response Payload]

Suggested caption: Figure 4.4. Raw JSON response payload from GET /api/v1/history?asset_id=... demonstrating provenance metadata attributes.

Confidentiality note: Anonymize customer asset UUIDs.

4.10 Endpoint Investigation UI Validation — A9 Gate

The A9 validation gate represents the primary end-to-end integration checkpoint of the EIMS project. This milestone verifies that operational evidence persisted in the backend database is delivered and rendered accurately within the Next.js web dashboard.

Verification Criteria and Observed Results

During live browser testing on the operational dashboard (/endpoints/[asset_id]):

Evaluation Criterion	Expected Behavior	Observed Result	Gate Verdict
Event Evidence Visibility	Dashboard renders operational event table	Visible: 49 persisted events rendered with severity tags, timestamps, and provider names.	PASS
Channel Scoping	Display reflects System and Application events	Confirmed: 34 System and 15 Application events displayed with active filter toggles.	PASS
AI Findings Panel	Dashboard displays synthesized diagnostic findings	Visible: Structured diagnostic cards rendered inline, displaying event descriptions, AI summaries, and remediation steps.	PASS
Empty State Absence	"No AI findings yet" placeholder hidden when data exists	Confirmed: The placeholder was not shown; active findings populated the	PASS

		container.	
Filtered API Status	Client fetch requests execute successfully	Confirmed: Network console recorded HTTP 200 OK for /api/v1/history?asset_id=....	PASS
Overall A9 Verdict	All criteria satisfied simultaneously	PASS	PASS

The passing of the A9 gate definitively verified the downstream data pipeline: from database persistence through backend API routing to client-side React rendering.

[FIGURE PLACEHOLDER — A9 UI Gate: Endpoint AI Findings and Event Evidence View]

Suggested caption: Figure 4.5. Browser runtime verification (A9 Gate) showing simultaneous rendering of Windows Event Evidence and inline AI Findings for ASSET-01.

Confidentiality note: Redacted screenshot from private archive; all real enterprise identifiers masked.

4.11 Supporting Search and Observability Results

In addition to the primary downstream validation workflow, supporting architectural subsystems were inspected to verify baseline operational health.

Core Service Health

Inspection of container health endpoints (GET /api/v1/health) and observability metrics confirmed:

- PostgreSQL 16: UP (Database responsiveness verified, active connections managed via PgBouncer).

- Redis 7: UP (In-memory broker responding with sub-millisecond latency).

- MinIO S3: UP (Storage buckets initialized and accessible for document ingestion).

Status of Secondary Supporting Capabilities

To maintain academic transparency, the operational testing status of secondary capabilities during the latest on-site validation session is formally recorded:

- Global Search (Ctrl+K): Basic keyboard shortcut navigation and API endpoint responsiveness were confirmed in unit testing; however, comprehensive multi-domain keyword stress testing was not tested in detail during this field session.

- Unified Timeline: The timeline API endpoint (GET /api/v1/timeline) was verified structurally, but chronological cross-domain aggregation was not tested in detail during this field session.

- Sticker OCR Ingestion: Multipart upload endpoints and MinIO storage commits were verified in automated test suites; physical chassis sticker scanning was not tested during this field session.

These supporting features remain functional prototype components, while the core downstream validation workflow represents the verified operational path.

4.12 Defects Identified and Corrected During Validation

Iterative testing and field deployment identified several technical defects that were systematically investigated, diagnosed, and resolved.

Defect Identifier &	Detection Mechanism	Root Cause Analysis	Engineering	Post-Fix Verification

Title			Remediation	
DEF-01: Responsive Sidebar Hydration Mismatch	Browser console runtime error during page load.	The sidebar layout component evaluated browser viewport width (window.innerWidth) during initial rendering, producing different HTML on the server runtime versus the client DOM.	Refactored responsive layout logic to rely on deterministic CSS media queries and added client-side hydration mounting guards.	Next.js production build and browser rendering confirmed zero hydration mismatch warnings.
DEF-02: Filtered History Route HTTP 500 Error	API integration test and field validation of /api/v1/history?asset_id=....	The router function signature utilized the Optional type annotation for the query parameter, but Optional had not been imported from the standard Python typing library.	Added Optional to the typing module imports in the history router controller.	Endpoint returned HTTP 200 OK across both filtered and unfiltered queries.
DEF-03: Environment Dataset Discrepancy	Local developer testing in home environment.	Querying the history API returned zero records for ASSET-01 when executed on a clean developer machine.	Investigation confirmed this was not an application bug, but the expected result of environment isolation: the developer database lacked the field dataset present on the authorized work machine.	Verified documentation standards and confirmed dataset provenance boundaries across environments.
DEF-04: Historical Benchmark Data-Loss Incident	Historical database audit during early engineering sprint.	A destructive benchmark script was executed against the primary database container, exposing insufficient isolation between benchmarking routines and real operational data stores. Original asset records were lost.	Implemented strict separation of validation environments, permanently prohibited running destructive benchmark scripts in standard workflows, and introduced safety guards.	Verification protocols now strictly utilize non-destructive validation suites; surviving real sources (MinIO objects, USB reports) were preserved.

Documenting these defects explains the technical issues encountered and resolved during development.

4.13 Evidence Handling and Confidentiality

To satisfy corporate non-disclosure obligations while maintaining academic traceability, all evidence captured during testing was governed by a strict data handling protocol:

1. Physical Separation of Raw Artifacts: Raw field evidence—including unredacted JSON reports, raw operating system event dumps, and full-resolution unredacted browser screenshots—is stored in a private evidence workspace located entirely outside the public Git repository.

2. Repository Anonymization: No customer hostnames, enterprise IP subnets, MAC addresses, hardware serial numbers, employee account names, or internal Active Directory domain names were committed to version control.

3. Use of Academic Aliases: All documentation, reports, and presentation slides consistently utilize standardized aliases (ASSET-01, Windows Server A).

4. Exclusion of Cryptographic Secrets: In accordance with the BitLocker secret boundary rule, zero recovery keys or plaintext passwords were collected, stored, or exposed.

5. Sanitization of Visual Evidence: Screenshots intended for academic defense or thesis publication undergo strict visual redaction, obscuring IP addresses, server names, and internal path structures.

4.14 Validation Limitations

To ensure objective and defensible academic reporting, the limitations of the validation activities must be explicitly stated:

1. Downstream Live Chain Focus: The latest on-site validation session verified the downstream data chain (persisted database rows, API routing, and web UI rendering). It did not re-execute the upstream USB audit collection or re-generate original JSON reports.

2. Absence of Live Duplicate Re-Testing: The sequential deduplication algorithm was verified in automated test suites and earlier testing sessions; it was not re-tested with duplicate live report uploads during the final on-site session.

3. Application-Level Deduplication Boundary: Event deduplication is enforced sequentially in application logic. It does not provide database-level concurrency guarantees against simultaneous parallel uploads.

4. Windows-Centric Scope: Field validation was conducted exclusively on Microsoft Windows operating systems. Linux distributions and macOS endpoints were not evaluated.

5. Bounded Analytical Workload: The AI Analyzer was evaluated against a bounded batch size of 10 events. System behavior under catastrophic log flooding exceeding hundreds of simultaneous critical failures was not evaluated.

6. Assistive Nature of AI: Artificial intelligence findings were evaluated as diagnostic triage assistance; they were not assessed as autonomous or legally binding root-cause determinations.

7. Single-Node Prototype Infrastructure: Validation was conducted on single-node container infrastructure. High-availability clustering, automated PostgreSQL failover, and distributed Kubernetes deployments were not evaluated.

8. Selective Secondary Feature Testing: Supporting features—specifically Global Search, Unified Timeline, and Sticker OCR—were verified at the component level but were not subjected to comprehensive field validation during the final session.

4.15 Validation Summary

The matrix below consolidates the empirical findings across all evaluated engineering domains:

Engineering Validation Domain	Verification Methodology	Observed Result	Operational Status
Ingestion Regression Suite	Automated pytest execution	37 Passed, 0 Failed	PASS

	(37 tests)		
Frontend Static Typing	TypeScript compiler (tsc --noEmit)	Exit Code 0, 0 Errors	PASS
Production Bundle Compilation	Next.js production build (npm run build)	Exit Code 0, Clean Build	PASS
Asset Registry Ingestion	Database inspection for ASSET-01	Entity created, specs populated	PASS
Operating System Event Persistence	API query (GET /api/v1/telemetry/winlogs)	49 rows persisted (System/App)	PASS
Sequential Deduplication	Automated test suite & previous field testing	Duplicate events filtered (new=0)	PREVIOUSLY VERIFIED
Severity Prioritization	Queue sorting & analysis selection	Critical/Error prioritized	PASS
AI Analyzer Workload Bounding	Analysis record count inspection	Bounded to 10 events	PASS
Diagnostic Provenance	Metadata inspection in analysis_history	source_type & event_source_id linked	PASS
Filtered History API	HTTP request (GET /api/v1/history?asset_id=...)	HTTP 200 OK (10 filtered rows)	PASS
A9 Browser UI Gate	Interactive browser verification on /endpoints/[id]	Event table & AI findings visible	PASS
Core Subsystem Health	/api/v1/health diagnostic probes	PostgreSQL, Redis, MinIO UP	PASS
Global Search Palette	Component testing	Keyboard navigation responsive	NOT TESTED IN DETAIL
Unified Timeline	Endpoint query verification	Schema structure valid	NOT TESTED IN DETAIL
Sticker OCR Pipeline	Automated integration tests	MinIO upload & text extraction valid	NOT TESTED IN DETAIL
Upstream Reacquisition in Final Session	Field operational protocol	Final session inspected downstream data	NOT RE-RUN IN LATEST SESSION

In conclusion, the experimental and field validation demonstrated the functional integrity, data provenance, and user interface delivery of the primary downstream validation workflow. The platform successfully bridges air-gapped field evidence collection with centralized relational persistence and AI-assisted operational triage.

Source docs/report/CHAPTER_5_SUMMARY_AND_RECOMMENDATIONS.md

Chapter 5

Summary and Recommendations

Current graduation revision and evidence boundary

Current source audit: 1 November 2026 — 2a0d9031297ac1063a0f327a8a76c455bb5da315 (PR #10). Next.js 16.3.0 / React 19.2.8. Engineering CI: 391 Python tests (299 Analyzer hermetic + 46 synthetic PostgreSQL + 46 USB isolated), run 41230498604, not project-wide certification. DEV 12 DRAFT, HELDOUT 5 reserved, claims 61, genuine human reviews 0. Diagnostic accuracy NOT_MEASURED; secure HTTPS browser BLOCKED_NOT_TESTED; current USB/production acceptance NOT_TESTED. No main merge.

The release is ENGINEERING_RC_WITH_EXTERNAL_GATES, not a completed production deployment. Current contributions are vendor-aware deterministic investigation, source-linked evidence, conservative uncertainty, secure owner-scoped History, and reproducible evaluation/review tooling. The most important remaining experiment is authentic review of fort_config, esxi_tracker and win_policy, with claim judgments bound to exact prediction hashes. A three-case pilot supports workflow verification only, not broad accuracy. Next steps are approved HTTPS acceptance, source-aligned academic artifact rebuild and owner-authorized core merge (#2→#3→#4→#5→#10) with CI after retargeting. Infrastructure #6→#7→#8→#9→#11 stays separate; four recovery XFAIL contracts are not resolved by the core CI. New vendors, RAG/LLM, cloud migration and full tenancy are deferred. Earlier prototype conclusions below retain their historical scope only.

Historical prototype material (retained for provenance)

This chapter summarizes the results, limitations, and lessons learned of the Enterprise Infrastructure Management System (EIMS) project. It evaluates the completed platform against the engineering objectives established in Chapter 1, consolidates key technical outcomes and validation evidence, analyzes engineering challenges and lessons learned, documents current system limitations, and outlines future architectural recommendations based on the project roadmap.

5.1 Project Summary

The Enterprise Infrastructure Management System (EIMS) was designed and implemented as an infrastructure evidence management and AI-assisted investigation platform. Modern enterprise computing environments frequently feature distributed, heterogeneous computing estates spanning secure subnets, demilitarized zones, and air-gapped networks. In these environments, system administrators and infrastructure engineers face substantial operational friction: hardware and configuration records are fragmented across unlinked spreadsheets, system logs are locked inside local binary stores, and continuous outbound connectivity to centralized cloud monitoring platforms is often restricted by enterprise security policies.

To address these challenges, EIMS was architected to bridge the operational gap between connected and disconnected endpoints without imposing intrusive dependencies on audited systems. The completed graduation baseline provides:

- A normalized relational Asset Registry cataloging hardware specifications, network configurations, and operational lifecycle states.

- A zero-dependency Portable USB Auditor equipped with an embedded CPython 3.14.3 amd64 runtime, capable of interrogating Windows endpoints via read-only interfaces and generating local JSON evidence reports.

- An Offline Ingestion Pipeline that parses multipart JSON reports, executes relational upserts, and persists operating system events into structured datastores.

- An Application-Level Sequential Deduplication Engine utilizing SHA-256 composite hashing to prevent duplicate record insertion during repeated report imports.

- A Severity-Ranked Prioritization Mechanism that filters high-volume operational event streams, routing critical anomalies to downstream triage.

- A bounded AI Log Analyzer utilizing local semantic vector embeddings (FastEmbed BAAI/bge-small-en-v1.5 primary, with SentenceTransformer all-MiniLM-L6-v2 fallback) and PostgreSQL pgvector similarity search to synthesize contextual diagnostic summaries and mitigation recommendations.

- Persistent Audit Provenance Tracking embedding source classifications (USB_OFFLINE_COLLECTION) and event linkage identifiers directly into analysis history records.

- An interactive Operational Web Dashboard built with Next.js, React, and TypeScript, delivering endpoint inspection views, security posture meters, universal keyboard search (Ctrl+K), and unified operational timelines.

- Supporting capabilities for Hardware Manifest OCR Registration backed by S3-compatible object storage (MinIO), alongside standardized containerized observability.

EIMS is specifically positioned as an operational triage and evidence management system. It does not function merely as a static dashboard, nor does it attempt to replicate the vast enterprise-wide log indexing of commercial SIEM platforms or the specialized evidentiary standards of certified digital forensics software. Instead, it provides a lightweight, cohesive engineering prototype that unifies asset tracking, air-gapped evidence collection, and AI-assisted operational triage.

5.2 Achievement of Project Objectives

The success of the EIMS project is evaluated against the seven concrete engineering objectives established in Section 1.3 of this report. The matrix below maps each objective to its realized implementation outcome and empirical validation evidence:

#	Project Objective (Chapter 1)	Implementation Outcome	Empirical Validation Evidence (Chapter 4)	Achievement Status
1	Architect and Implement a Centralized Asset Registry: Design a normalized relational model and API gateway indexing assets, hardware, network, and lifecycle states.	Implemented PostgreSQL relational schema (infrastructure_assets, hardware_inventories) with UUIDv4 canonical identifiers, state machine validation, and FastAPI	Verified in live database inspection and API testing; ASSET-01 successfully indexed with hardware specifications, network IPs, and compliance state.	ACHIEVED

		administrative CRUD endpoints.		
2	Develop a Lightweight, Portable Endpoint Collection Utility: Implement a portable auditing package with embedded runtime executing on Windows hosts without preinstalled dependencies.	Constructed Portable USB Auditor (clients/usb_auditor/) featuring embedded CPython 3.14.3 amd64 runtime, batch launcher (Run-EIMS-Audit.bat), read-only CIM queries, and package manifest verification.	Earlier on-site field testing confirmed successful execution and local JSON evidence generation on Windows Server without host software installation.	ACHIEVED
3	Establish an Offline Ingestion and Normalization Pipeline: Construct backend API endpoints parsing offline multipart reports and persisting operating system events.	Developed canonical ingestion endpoint POST /api/v1/assets/import-report accepting multipart JSON payloads, updating asset entities, and committing events to windows_event_logs.	Field verification confirmed HTTP 200 ingestion response; 49 operating system events persisted in relational database linked to ASSET-01.	ACHIEVED
4	Implement Deterministic Sequential Deduplication and Severity Prioritization: Design application-level SHA-256 fingerprinting to prevent redundant records, coupled with severity-ranked priority queues.	Implemented composite SHA-256 hashing across five canonical event attributes, JSONB query filtering, and 4-tier severity priority sorting (Critical > Error > Warning > Information).	Ingestion regression suite (37 tests passed) confirmed duplicate detection (new=0, duplicate=N); field data confirmed priority-ranked candidate selection.	ACHIEVED
5	Integrate an AI-Assisted Log Investigation Engine: Incorporate local sentence embeddings and vector similarity search to produce structured diagnostic summaries.	Built AI Analyzer service utilizing local FastEmbed (BAAI/bge-small-en-v1.5 primary, with SentenceTransformer all-MiniLM-L6-v2 fallback, 384 dimensions) and PostgreSQL pgvector cosine distance retrieval against ai_knowledge, bounded to max 10 events per run.	Field query confirmed 10 analysis history records linked to ASSET-01; diagnostic summaries and mitigation recommendations populated.	ACHIEVED
6	Deliver an Interactive Web-Based Operational	Built Next.js operational dashboard (clients/dashboard/)	Browser runtime verification (A9 Gate) passed; dashboard	ACHIEVED

	Dashboard: Develop a responsive web interface rendering asset details, compliance postures, event tables, AI findings, search, and timelines.	featuring App Router architecture, endpoint detail cards, event viewer, inline AI findings, Ctrl+K search, and unified timelines.	successfully rendered 49 event records and inline AI findings for ASSET-01.	
7	Validate System Reliability and Integrity Through Automated and Field Testing: Verify platform stability, data provenance, and workflows using automated test suites and real-world field validation.	Executed multi-tier testing combining unit/integration test suites (pytest), TypeScript compilation, Next.js production builds, and on-site field verification on enterprise Windows Server hosts.	37 ingestion tests passed (0 failures); TypeScript passed (Exit 0); Next.js build passed (Exit 0); downstream live field chain verified. *(Note: Latest session verified downstream persistence rather than re-executing full USB acquisition).*	ACHIEVED

All seven foundational engineering objectives defined at the inception of the cooperative education project were successfully fulfilled within the established graduation scope.

5.3 Key Engineering Outcomes

The realization of EIMS yielded six significant engineering outcomes that address the operational challenges of infrastructure management:

1. Unified Infrastructure Evidence Representation

EIMS establishes a centralized relational anchor for enterprise compute hosts. Rather than maintaining isolated spreadsheets and disparate log directories, the platform consolidates hardware component configurations, operating system build metadata, security baseline compliance scores, and diagnostic event logs under a canonical asset entity (infrastructure_assets).

2. Zero-Dependency Portable Evidence Collection

The design and implementation of the Portable USB Auditor demonstrated the viability of zero-dependency auditing on production Windows systems. By bundling an embedded CPython 3.14.3 amd64 runtime directly on removable media and initiating scans via native batch scripts, the utility interrogates target hosts through native read-only CIM cmdlets without requiring persistent background daemons, administrative software installation, or runtime compilers on audited servers.

3. Traceable Event-to-Analysis Provenance Chain

A central engineering achievement of EIMS is traceable data provenance. The platform establishes an explicit linkage connecting raw field evidence to high-level diagnostic findings:

Target Host -> [Audit] -> Local JSON -> [Ingest] -> WindowsEventLog -> [Triage] -> AnalysisHistory -> [Render] -> Dashboard UI

Every analytical record persists explicit provenance metadata inside analysis_history.event_metadata:

- source_type = "USB_OFFLINE_COLLECTION"

- asset_id (binding the finding to the parent compute asset)

- event_source_id (JSON attribute linking directly to windows_event_logs.log_id)

- _reporter_dedup (cryptographic hash ensuring deduplication traceability)

This enables operators and compliance auditors to inspect an AI-generated remediation suggestion and trace it backward through the database to the exact event log record and host that generated it.

4. Application-Level Duplicate Protection

Periodic offline auditing inevitably produces overlapping time-window reports. EIMS resolves this through deterministic application-level deduplication, computing a composite SHA-256 hash across the asset identifier, channel, provider, record identifier, and event's recorded timestamp. During sequential imports, identical events are recognized and filtered in memory, preventing database bloat and eliminating redundant analytical computation while acknowledging known concurrency boundaries.

5. Workload-Bounded AI Diagnostic Assistance

By combining deterministic severity prioritization with local vector similarity retrieval (pgvector cosine distance), the AI Analyzer isolates critical errors and warnings from noisy event streams. Capping analytical workloads at a maximum of 10 events per import run guarantees predictable resource consumption and prevents database thread starvation, positioning artificial intelligence as a practical, bounded triage aid for systems engineers.

6. Real-World Field Validation on Production Infrastructure

The platform was deployed and evaluated against authorized enterprise infrastructure, validating the end-to-end ingestion and presentation pipeline using genuine operational event logs extracted from an enterprise Windows Server host (ASSET-01).

5.4 Validation Summary

The experimental and field validation results detailed in Chapter 4 are summarized in the verification matrix below:

Architectural Domain	Verification Method	Measured Field Result	Operational Status
Ingestion Regression Suite	Automated pytest execution	37 passed, 0 failed, 4 warnings	PASS
Frontend Static Typing	TypeScript compiler (tsc --noEmit)	Exit Code 0, 0 type errors	PASS
Production Bundle Compilation	Next.js production build (npm run build)	Exit Code 0, clean build	PASS
Asset Registry Entity Persistence	Relational database inspection	ASSET-01 entity created with complete specs	PASS
Operating System Event Evidence	Filtered API query (/api/v1/telemetry/winlogs)	49 rows persisted (34 System, 15 Application)	PASS
AnalysisHistory Asset Linkage	Filtered History API (/api/v1/history?asset_id=...)	HTTP 200 OK (10 filtered records for asset)	PASS
Analytical Workload Bounding	Analysis record count evaluation	Bounded to exactly 10 events	PASS

Audit Provenance Verification	Metadata inspection in analysis_history	source_type & event_source_id linked	PASS
A9 Browser Runtime UI Gate	Browser inspection on /endpoints/[asset_id]	49 events visible, inline AI findings rendered	PASS
Core Subsystem Health	/api/v1/health diagnostic probes	PostgreSQL, Redis, MinIO operational (UP)	PASS
Sequential Deduplication	Automated test suite & previous field tests	Redundant events filtered without error	PREVIOUSLY VERIFIED
Full Upstream Reacquisition	Operational validation protocol	Downstream persistence inspected in latest session	NOT RE-RUN IN LATEST SESSION
Secondary Features (Search/Timeline/OCR)	Component-level integration tests	Core endpoints functional; not field-tested in depth	NOT TESTED IN DETAIL

The validation confirmed the functional reliability and data integrity of the primary downstream downstream validation workflow, demonstrating that offline field evidence can be successfully ingested, deduplicated, analyzed, and visualized.

5.5 Engineering Challenges and Lessons Learned

The design and implementation of EIMS encountered several complex technical challenges. Documenting the engineering resolutions provides valuable lessons for enterprise software engineering:

1. Server-Side Rendering (SSR) and Client Hydration Discrepancies

During early frontend development, the responsive sidebar layout component evaluated browser viewport width (window.innerWidth) during initial rendering. Because browser window dimensions do not exist within the server runtime, the HTML generated by the server differed from the initial DOM constructed by the client browser, triggering React hydration mismatch warnings and visual flickering.

- Engineering Lesson: In hybrid Next.js/React applications, responsive visibility must rely on deterministic CSS media queries rather than runtime JavaScript branching during initial component render, supplemented by defensive client-side hydration mounting guards.

2. Framework Annotation Typing Bugs in Dynamic Routers

During field validation of the filtered history API, requests targeting /api/v1/history?asset_id=... failed with HTTP 500 internal server errors. Investigation revealed that the route function signature utilized the Optional type annotation for query parameter parsing, but Optional had been omitted from the typing imports in that specific controller file. Python evaluates type annotations dynamically; the missing Optional import in typing (from typing import List, Optional, commit acb855f) caused runtime request handling errors.

- Engineering Lesson: Python type hints in modern frameworks like FastAPI are evaluated dynamically at runtime by Pydantic; comprehensive API integration testing with realistic query parameter permutations is mandatory to catch missing type imports before deployment.

3. Dataset Provenance and Multi-Environment Discrepancies

During transition testing between developer workstations and field machines, querying the history API returned zero records for ASSET-01 on clean development containers, initially raising concerns of a persistence regression. Investigation confirmed that the application codebase was behaving correctly, but the isolated developer database simply lacked the field dataset populated during on-site server testing.

- Engineering Lesson: Software verification must strictly differentiate between code defects and dataset presence; automated integration testing must rely on hermetic database seed fixtures to ensure deterministic test execution across disparate physical environments.

4. Database Safety Controls and Benchmark Isolation

During stress testing, executing a destructive benchmark against the development database caused data loss in that instance, reinforcing the need for strict database isolation guards and separate testing fixtures.

- Engineering Lesson: Benchmarking and load-testing scripts must be structurally decoupled from operational database configurations; destructive testing commands must be permanently barred from standard deployment workflows, and persistent volumes must enforce strict separation to safeguard real engineering evidence.

5. Multi-Tier Evidence-Driven Verification

Early validation relied heavily on inspecting backend console logs. However, true end-to-end reliability was only achieved when validation protocols required cross-layer assertions: verifying database state in PostgreSQL, asserting HTTP status codes and JSON payloads at the API boundary, and inspecting DOM elements and network waterfalls in live browser runtimes (culminating in the A9 Gate).

- Engineering Lesson: In multi-tier platforms, verifying backend logs alone is insufficient; defensible validation requires end-to-end verification that asserts data consistency across persistence, API transport, and user interface rendering layers.

5.6 Current System Limitations

To maintain academic rigor and transparent engineering reporting, the boundaries and limitations of the current EIMS graduation release are explicitly documented:

1. Windows-Centric Scope: The current implementation of the Portable USB Auditor and Windows Event Log parser is specialized exclusively for Microsoft Windows operating systems (Windows Server 2022/2019, Windows 11/10). Telemetry collection on Linux distributions or macOS endpoints is not supported.

2. Channel-Bounded Event Ingestion: The portable audit workflow intentionally scopes event log acquisition to the System and Application channels. Security event collection is omitted from this workflow.

3. Application-Level Deduplication Boundary: Event deduplication is enforced sequentially within application services. Because PostgreSQL does not enforce unique constraints across the JSONB deduplication path, concurrent parallel uploads of identical reports across multiple threads could experience race conditions.

4. Bounded Analytical Capacity: The AI Analyzer is strictly capped at processing at most 10 prioritized events per batch import. While this bounds computational overhead and controls response latency, events outside the selected subset are not automatically analyzed by re-importing the same report; a separate review or reanalysis workflow would be future work.

5. Assistive Nature of Artificial Intelligence: Diagnostic summaries, root-cause analyses, and mitigation suggestions generated by the AI Analyzer represent probabilistic recommendations derived from semantic vector retrieval; they do not constitute infallible, legally binding, or autonomous root-cause determinations.

6. Non-Destructive Read-Only Operation: EIMS deliberately omits automated host remediation capabilities. The platform identifies misconfigurations and suggests remediation steps, but does not execute automated configuration changes or service restarts on monitored endpoints.

7. Operational Triage vs. Certified Forensics: EIMS is designed as an infrastructure management and operational triage aid. It is not designed, validated, or certified as a court-admissible digital forensics acquisition utility.

8. Single-Node Prototype Deployment: The current graduation baseline is deployed via single-node container infrastructure orchestrated with Docker Compose; it does not implement enterprise high-availability clustering, distributed multi-region failover, or enterprise Single Sign-On (SSO) federation.

9. Selective Secondary Feature Field Testing: While the primary downstream validation workflow was thoroughly verified on-site, secondary capabilities—specifically Global Search, Unified Timeline, and Sticker OCR—were validated primarily in automated component suites rather than under comprehensive field conditions.

5.7 Recommendations and Future Work

Based on the operational findings and the long-term engineering vision documented in the EIMS Project Roadmap (ROADMAP.md), future development should focus on six strategic milestones:

1. Concurrency-Safe Deduplication and Relational Hardening

To eliminate the potential for race conditions during simultaneous parallel report uploads, future releases should enhance the deduplication architecture:

- Introduce database-level functional unique indexes across the JSONB deduplication hash (CREATE UNIQUE INDEX idx_winlogs_dedup ON windows_event_logs ((evtx_metadata->>'_reporter_dedup'))).

- Implement distributed advisory locks in PostgreSQL or Redis to serialize concurrent ingestion requests targeting the same physical asset.

2. High Availability and Enterprise Scale Production Clustering (Roadmap Milestone 1)

As outlined in Milestone 1 of the Project Roadmap, production deployment requires transitioning from single-node containers to a resilient, clustered infrastructure:

- Database High Availability: Deploy a multi-node PostgreSQL read-replica cluster with streaming replication, utilizing PgBouncer for connection pooling alongside separately configured routing and failover mechanisms.

- Distributed Caching & Brokering: Transition Redis to a Redis Sentinel or Redis Cluster topology to ensure broker partition tolerance and high availability.

- Container Orchestration: Develop production Kubernetes (K8s) deployment manifests and Helm charts, incorporating Horizontal Pod Autoscaling (HPA) to scale telemetry ingestion workers dynamically under burst traffic.

- Automated Disaster Recovery: Implement automated point-in-time recovery (PITR) pipelines with encrypted off-site backup replication.

3. Cross-Endpoint Anomaly Correlation (Roadmap Milestone 2)

While EIMS currently analyzes operational events on a per-endpoint basis, advanced enterprise operations require correlated multi-host intelligence:

- Implement multi-host incident timeline reconstruction to track distributed operational cascades and multi-tier service outages across networked servers.

- Develop graph-based dependency visualization mapping network connections and service dependencies between monitored infrastructure assets.

4. Enterprise Identity Federation and Granular RBAC (Roadmap Milestone 2)

To facilitate integration into corporate enterprise environments:

- Implement Enterprise Single Sign-On (SSO) supporting SAML 2.0 and OpenID Connect (OIDC) protocols to integrate with corporate identity providers (such as Microsoft Entra ID and Okta).

- Refine Role-Based Access Control (RBAC) to support granular, tenant-scoped organizational permissions and administrative approval workflows.

5. Multi-Platform Endpoint Expansion (Possible Future Extension)

While EIMS currently focuses on Windows-dominated enterprise infrastructure, possible future extensions beyond the graduation prototype could examine multi-platform support:

- Explore a portable Linux auditing package capable of collecting systemd journal logs, hardware configurations via /sys and /proc, and local security postures without requiring preinstalled dependencies.

- Extend normalization schemas to ingest heterogeneous syslog and journald telemetry into the centralized asset registry.

6. Rigorous Long-Duration Operational Validation

Future validation efforts should expand upon the cooperative education baseline:

- Conduct long-duration soak testing under continuous high-frequency telemetry ingestion to evaluate database index leaf fragmentation and memory stability over weeks of sustained operation.

- Execute formal third-party vulnerability assessments and penetration testing to validate container isolation and API gateway security boundaries.

5.8 Overall Conclusion

The Enterprise Infrastructure Management System (EIMS) project successfully demonstrated the engineering feasibility of combining centralized infrastructure asset management with portable offline evidence collection and AI-assisted operational triage within a cohesive, lightweight platform architecture.

By confronting the concrete operational realities of enterprise IT—specifically the prevalence of air-gapped, segmented, or policy-restricted Windows endpoints where persistent daemons cannot be deployed—the project established a safe, non-destructive evidence acquisition methodology using self-contained portable media. The implementation proved that operational telemetry and operating system event logs can be extracted without host software prerequisites, ingested into normalized relational datastores, protected against duplicate insertion via cryptographic fingerprinting, and prioritized for diagnostic analysis. Furthermore, the integration of local semantic vector embeddings (FastEmbed BAAI/bge-small-en-v1.5 primary, with SentenceTransformer all-MiniLM-L6-v2 fallback) and PostgreSQL pgvector similarity search validated the practical utility of Retrieval-Augmented Generation (RAG) in providing context-grounded mitigation advice while maintaining strict audit provenance back to raw event records.

Through rigorous automated testing and empirical field validation on authorized enterprise Windows Server infrastructure, the platform proved that modern web visualization can deliver actionable operational visibility across complex infrastructure estates. While enterprise clustering, distributed orchestration, and cross-

endpoint correlation remain valuable targets for post-graduation evolution, the completed EIMS graduation baseline stands as a fully functioning, structured, and robust engineering foundation for modern infrastructure evidence management.

References

[1] AXELOS, *ITIL Foundation: ITIL 4 Edition*. London, UK: TSO (The Stationery Office), 2019. ISBN: 978-0-11-331607-6.

[2] Microsoft Corporation, "Windows Management Instrumentation Architecture," *Microsoft Learn*, May 2021. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/wmisdk/wmi-architecture. Accessed: Sep. 29, 2026.

[3] Microsoft Corporation, "About the Windows Event Log," *Microsoft Learn*, Jan. 2021. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/wes/about-the-windows-event-log. Accessed: Sep. 29, 2026.

[4] P. Buneman, S. Khanna, and W.-C. Tan, "Why and where: A characterization of data provenance," in *Database Theory - ICDT 2001 (Lecture Notes in Computer Science)*, vol. 1973, J. Van den Bussche and V. Vianu, Eds., London, UK: Springer Berlin Heidelberg, 2001, pp. 316–330. doi: 10.1007/3-540-44503-X_20.

[5] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, "A Survey on Automated Log Analysis for Reliability Engineering," *ACM Computing Surveys*, vol. 54, no. 6, art. 130, pp. 1–37, Jul. 2021. doi: 10.1145/3460345.

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3982–3992. doi: 10.18653/v1/D19-1410.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS 2020)*, vol. 33, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, Eds., 2020, pp. 9459–9474.

[8] PostgreSQL Global Development Group, "PostgreSQL 16.0 Documentation," *PostgreSQL Documentation*, Sep. 2023. [Online]. Available: https://www.postgresql.org/docs/16/index.html. Accessed: Sep. 29, 2026.

[9] A. Kane, "pgvector: Open-source vector similarity search for Postgres," *pgvector Documentation*, 2023. [Online]. Available: https://github.com/pgvector/pgvector. Accessed: Sep. 29, 2026.

[10] Redis Ltd., "Redis Documentation," *Redis Documentation*, 2024. [Online]. Available: https://redis.io/docs/latest/. Accessed: Sep. 29, 2026.

[11] MinIO Inc., "MinIO Object Storage Documentation," *MinIO Technical Documentation*, 2024. [Online]. Available: https://min.io/docs/minio/linux/index.html. Accessed: Sep. 29, 2026.

[12] R. Smith, "An Overview of the Tesseract OCR Engine," in *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, Curitiba, Brazil: IEEE, Sep. 2007, vol. 2, pp. 629–633. doi: 10.1109/ICDAR.2007.4378789.

[13] S. Ramírez, "FastAPI Documentation," *FastAPI*, 2024. [Online]. Available: https://fastapi.tiangolo.com/. Accessed: Sep. 29, 2026.

[14] Vercel Inc., "Next.js Documentation," *Next.js Documentation*, 2024. [Online]. Available: https://nextjs.org/docs. Accessed: Sep. 29, 2026.

[15] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," National Institute of Standards and Technology, Gaithersburg, MD, NIST Special Publication (SP) 800-207, Aug. 2020. doi: 10.6028/NIST.SP.800-207.

[16] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Engineering Task Force (IETF), RFC 7519, May 2015. doi: 10.17487/RFC7519.

[17] C. Sridharan, *Distributed Systems Observability: A Guide to Building Robust Systems*. Sebastopol, CA: O'Reilly Media, 2018. ISBN: 978-1-492-03344-8.

[18] Prometheus Authors, "Overview | Prometheus," *Prometheus Documentation*, The Linux Foundation, 2024. [Online]. Available: https://prometheus.io/docs/introduction/overview/. Accessed: Sep. 29, 2026.